

SWE-Review: Closing the Loop on Issue Resolution with Agentic Code Review

Ruoyu Wang^{1*} Jierun Chen^{2*} Shaowei Wang² Chaofan Tao² Sidi Yang³
Yuxin Jiang² Kim-Hui Yap¹ Lifeng Shang² Xiaohui Li² Haoli Bai^{2†}

¹NTU ²Huawei Technologies ³HKU
 Webpage  Code  Data  Plugin

Abstract

Coding agents increasingly generate pull requests (PRs) for real-world software issues, yet one-shot PR generation remains open-loop: the PR is proposed without systematic review, diagnosis, or revision. We introduce **SWE-Review**, a framework for closing this loop with agentic code review. Given an issue and an AI-generated PR, a reviewer agent explores the repository, decides whether the PR should be accepted, and provides structured feedback for revision. We evaluate this setting with our proposed **SWE-Review-Bench** to measure both review correctness and downstream revision usefulness. We further curate **SWE-Review-Traj** dataset to study broader applications of agentic review and fill the data-scarcity gap for open reviewer training. Experiments show that agentic review continuously improves PRs through a generate-review-revise loop, outperforms single-turn fixed-context review in both decision accuracy and resolve rate after revision, transfers beyond review to improve issue-resolution models, and enables effective and efficient test-time scaling. These results position agentic code review as a practical mechanism for moving AI coding agents from one-shot PR generation toward closed-loop issue resolution.

1 Introduction

AI coding tools have advanced from autocomplete assistants to autonomous agents that can propose pull requests (PRs) for real-world software engineering (SWE) issues. Yet one-shot PR generation remains open-loop: after an agent produces a candidate PR, there is often no reliable mechanism for determining whether the issue has been resolved or for diagnosing how the PR should be revised when it has not.

Code review, the practice of inspecting changes before they are merged, is the natural mechanism for closing this loop. A reviewer provides both a binary decision, accept or request changes, and a diagnosis of what is wrong and how to fix it. As AI-assisted PR submissions continue to grow, automated review is becoming increasingly important. However, code review capabilities remain less developed than those of other software engineering tasks such as issue resolution, and we still lack systematic benchmarks, reliable metrics, public training data, and end-to-end evaluations for understanding review’s role in a closed-loop issue-resolution pipeline.

To fill this gap, we present **SWE-Review**, a framework for closing the issue-resolution loop with agentic code review. Given a repository, an issue, and an AI-generated pull request, the reviewer explores the environment and produces two outputs: a binary accept/request-changes decision and a structured diagnosis that guides revision when the PR should not be merged. To study this setting

*Equal contribution

†Corresponding author: baihaoli@huawei.com

systematically, we construct **SWE-Review-Bench**, a benchmark of 1,384 candidate pull requests derived from 500 SWE-bench Verified issues and three PR generators spanning high-, medium-, and low-quality candidate distributions. We also construct **SWE-Review-Traj**, a dataset consisting of 8,914 agentic review trajectories to support training and evaluation of open reviewers. We evaluate review capacities with three complementary metrics: Completion Rate (CR), which measures whether the reviewer produces a parseable final review; Decision Accuracy (DA), which measures whether the reviewer makes the correct merge decision; and Resolve Rate after Revision (RRR), which measures whether the review improves the final patch outcome after revision. We will release the benchmark, review trajectories, and reviewer models to support reproducible research on open agentic code review.

Our experiments study agentic review in three complementary roles: as a reviewer for candidate PRs, as supervision for training open reviewers and issue-resolution models, and as verifier for test-time scaling. The results show that agentic review is useful throughout the issue-resolution pipeline:

- **Agentic reviews continually improve PRs through the iterative loop.** The generate-review-revise loop repeatedly improves candidate PRs on SWE-bench Verified, raising resolve rate from 27.5% to 56.9% for Qwen3-30B-A3B, from 50.9% to 68.8% for Qwen3-Coder-30B-A3B, and from 72.2% to 75.4% for GLM-5 (Figure 1).
- **Agentic review beats single-turn fixed-context review.** Across all three PR generator splits of our SWE-Review-Bench, agentic review outperforms single-turn fixed-context baselines in both DA and RRR (Figure 3), with the largest gains on harder tasks that require non-local repository reasoning (Figure 4).
- **Review trajectories improve issue resolution beyond review itself.** SFT on SWE-Review-Traj raises an 8B reviewer’s completion rate from approximately 4% to 71–84% and decision accuracy from near-random to 67–72% (Table 2); mixed training further improves direct resolve rate by up to 5.6 points and enables a unified generate-review-revise agent with up to 10.6 points final resolve-rate gain (Table 3).
- **Agentic review enables effective and efficient test-time scaling.** At test time, trained reviewers outperform dedicated verifiers in both effectiveness and efficiency, enabling review-guided iterative revision that raises resolve rate from 22.9% to 38.4% over only 4 revision rounds (Figure 5).

Together, these results position agentic code review as more than a post-hoc commenting tool. It is a practical mechanism for closing the issue-resolution loop: deciding whether AI-generated PRs should be accepted, producing feedback when they should not, and improving both the training and deployment of SWE issue-resolution agents.

2 Related Work

SWE issue resolution. Coding agents are emerging as a dominant paradigm for repository-level software issue resolution. Modern systems such as Cursor [5], Claude Code [3], Codex [23], Aider [10], and Devin [7] instantiate this paradigm through agent scaffolds [3, 25, 34, 37, 41, 45] that interact with development environments, invoke tools, and edit repositories to address real issues. This shift has also motivated a line of software-engineering benchmarks that evaluate agents in increasingly realistic, long-horizon, and repository-grounded settings [8, 13, 22, 44]. Alongside these benchmarks, recent work trains stronger issue-resolution agents by curating high-quality trajectories from reproducible execution environments [6, 12, 26, 29, 42]. While these works focus on improving PR generation, SWE-Review takes a complementary post-generation view. Rather than building another PR generator, we study the review mechanism needed after an AI-generated issue-resolution patch is produced: one that can filter out patches that fail to resolve the original issue and provide actionable diagnoses to support reliable revision.

Automated code review and evaluation. Automated code review aims to detect defects and produce review comments or accept/reject decisions for submitted changes. Existing systems formulate this as a single-turn task over a diff, sometimes augmented with retrieved context, producing labels or comments in one pass [9, 14–17, 20, 21, 28, 30, 33, 36]. This formulation assumes the diff contains sufficient evidence for judgment. For AI-generated issue-resolution patches, however, a seemingly correct diff may obscure the root cause, modify the wrong location, or introduce regressions

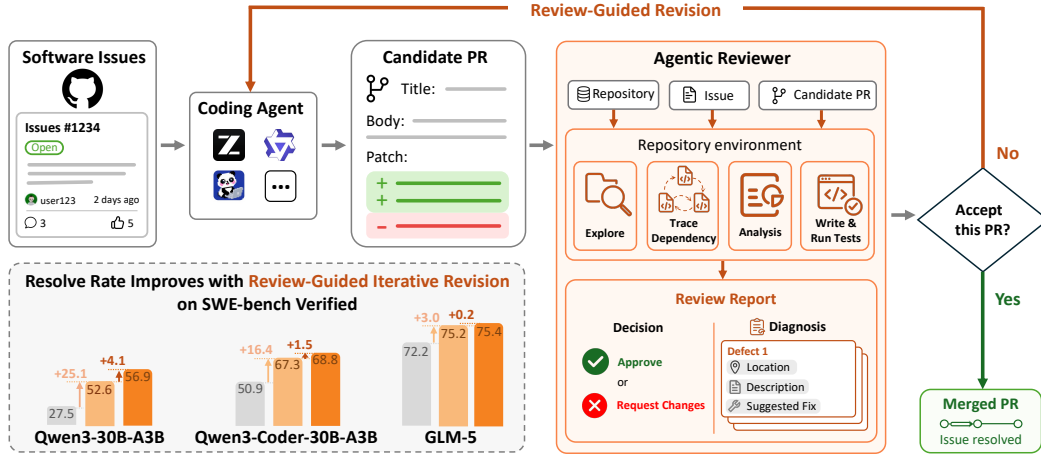


Figure 1: **SWE-Review turns one-shot PR generation into closed-loop issue resolution.** AI coding agents generate candidate PRs from real software issues; an agentic reviewer then inspects the repository, issue, and candidate PR in an executable environment before emitting a binary decision and structured diagnosis. Approved PRs are accepted as resolved, while rejected PRs are returned to the coding agent for review-guided revision. On SWE-bench Verified, this closed loop continuously raises the resolve rate across three PR generators, showing that agentic review is useful not only for judging PRs but also for improving them.

outside the edited hunks. Judging such patches requires repository-level evidence that cannot be pre-specified in a fixed context window: tracing call chains, checking related implementations, and verifying behavior. SWE-Review treats review as an agentic capability: the reviewer actively gathers evidence from the repository before committing to a decision and producing an actionable diagnosis. On the evaluation side, existing benchmarks compare generated comments against human references using text overlap, embedding similarity, or LLM-as-a-judge scores [11, 20, 27, 43, 46, 47], but these metrics do not directly measure whether the reviewer makes the right decision or provides feedback useful for revision. SWE-Review-Bench instead evaluates review end-to-end by measuring both decision correctness and downstream revision utility.

Test-time scaling and trained code scorers. Test-time scaling for issue resolution depends on selecting among candidate patches [32]. Prior work trains models to score candidate generation attempts: some generate an explicit verdict about whether the attempt is correct [12, 26, 31, 35], while others attach a learned scalar head to predict a ranking score [38]. In both cases, each attempt is reduced to a single score or decision used for ranking. Our reviewer can serve the same selection role, but its outputs are richer: the discrete decision supports ranking and early stopping at the first approved patch, while the structured diagnosis drives an iterative review-revision loop that scalar scorers cannot implement.

3 Agentic Code Review

AI coding agents have shown great promise for software issue resolution, where they generate candidate patches and submit pull requests (PRs) for integration. As AI-authored PRs are submitted at an unprecedented pace and scale, reliable automated review becomes essential for determining whether a patch resolves the issue and for providing feedback when it does not. We study agentic code review in this generate-review-revise loop. Section 3.1 formalizes the repository-grounded review task, introduces SWE-Review-Bench, and defines evaluation metrics. Section 3.2 tests whether agentic repository exploration improves over fixed-context single-turn review, and examines how this benefit varies with patch difficulty.

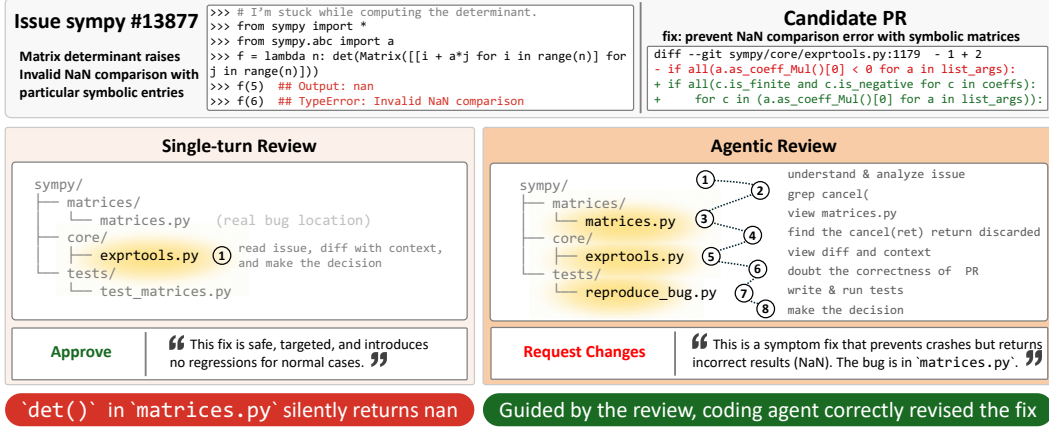


Figure 2: **Agentic review traces the bug across modules to its upstream root cause; single-turn approves a downstream symptom fix that silently returns wrong results.** On sympy-13877, the candidate PR adds a NaN guard in exprtools.py that prevents the crash but still leaves det() returning incorrect nan values. The agentic reviewer explores the repository, follows the call chain to matrices.py, identifies the discarded result of cancel(ret), and requests changes with an upstream fix suggestion. Full trajectory in Appendix D.

3.1 Task Formulation, Evaluation Metrics, and Our SWE-Review-Bench

We formalize agentic code review as a repository-grounded task. For each review instance, the input consists of three parts: a repository checkout at the relevant commit, a natural-language issue, and a candidate PR with its proposed diff, title, and body. The reviewer does not receive the golden patch or hidden test results. It may browse files, search code, inspect dependencies, and execute commands in the repository environment before submitting a review report. The review report contains two fields. The decision is binary: approve the PR if the patch resolves the issue, or request changes otherwise. When requesting changes, the reviewer also provides a diagnosis that identifies concrete defects, cites relevant code locations when possible, and proposes actionable fixes for a downstream revision agent. In this formulation, review serves as the pivot of the generate-review-revise loop shown in Figure 1.

Prior evaluations often score reviews against noisy or incomplete reference comments, or judge open-ended text with prompt-sensitive evaluators [16, 20, 27, 30, 36]. We instead evaluate the review report by its role in issue resolution with three metrics. Completion Rate (CR) measures whether the reviewer produces a parseable final review. Decision Accuracy (DA) compares the approve/request-changes decision against the patch’s true resolve status; incomplete reviews that produce no parseable decision are assigned a score of 0.5 (chance-level). Resolve Rate after Revision (RRR) evaluates diagnosis operationally as the final resolve rate after review and revision: approved patches are kept unchanged, while patches marked request-changes are returned to the original PR generator with the review feedback for one revision.

DA and RRR are defined operationally against executable verification of patch correctness; RRR additionally requires a standardized revision attempt conditioned on reviewer feedback. These definitions presuppose a SWE-style evaluation harness: grounded issues, hidden test-based verification of resolve status, and a revise-from-feedback procedure. Existing review benchmarks [11, 27, 43] rarely instantiate this executable revise-from-feedback loop end-to-end. We therefore construct SWE-Review-Bench based on SWE-bench Verified [22], a collection of real-world software issues with executable test suites. For the same 500 instances, we leverage the OpenHands-SDK [37] agent scaffold and three models of varying capability (GLM-5³ [1], Qwen3-Coder-30B-A3B [29], and Qwen3-30B-A3B [40]) to generate candidate PRs spanning high-, medium-, and low-quality distributions. After filtering PRs with empty patches, SWE-Review-Bench totals 1,384 PRs (GLM-5: 72.2% resolve rate, $n=500$; Qwen3-Coder-30B-A3B: 50.9%, $n=462$; Qwen3-30B-A3B: 27.5%,

³We use the locally deployed open-source GLM-5-FP8 model.

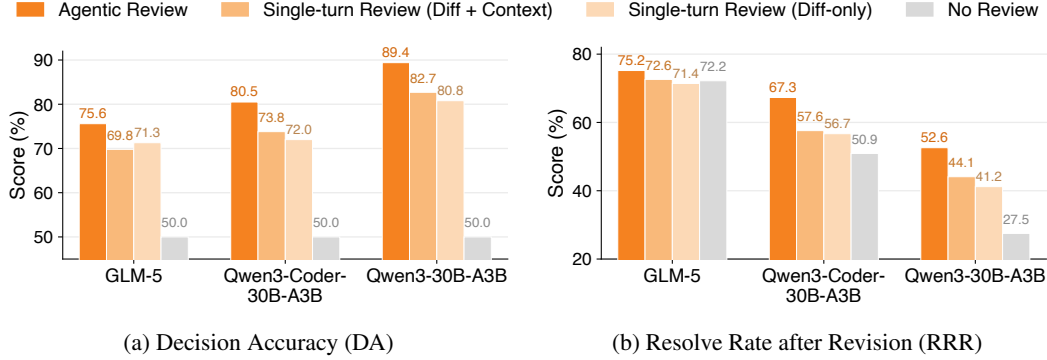


Figure 3: **Agentic review produces more accurate approve/request-changes decisions and more useful diagnoses that lead to higher resolve rates after revision than single-turn review.** PRs are generated by three models of varying capability (GLM-5, Qwen3-Coder-30B-A3B, and Qwen3-30B-A3B). All settings use the same reviewer model (Claude Opus 4.6 [4]), and completion rates are generally above 95% across settings.

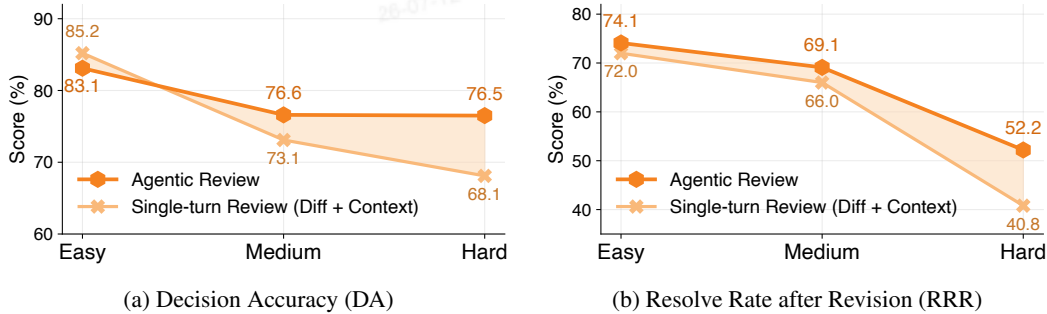


Figure 4: **Agentic review increasingly outperforms single-turn review as difficulty rises.** We stratify SWE-Review-Bench into easy, medium, and hard tertiles using the post-hoc difficulty criterion described in Section 3.2.

$n=422$). This stratification studies reviewer behavior under diverse candidate qualities rather than a single generator prior.

3.2 Agentic Review Better Guides PR Revision Than Single-Turn Review

With the task and benchmark fixed, we next isolate the value of interactive repository exploration. A cheaper alternative is single-turn review: present the issue, PR metadata, and diff to a strong model, optionally add retrieved files, and ask for a decision and diagnosis in one response [17, 28]. This baseline is important because it matches the interface of many current automated review systems and avoids the cost of multi-turn repository interaction. If fixed-context review were sufficient, an agentic reviewer would add complexity without clear benefit.

Single-turn review fixes the context before the reviewer knows what evidence is needed. This makes it brittle on non-local bugs: a PR can fix a symptom while leaving the root cause intact, depend on behavior outside the changed files, or violate conventions that are documented elsewhere. Agentic review instead gives the model an evidence-gathering procedure. It can inspect the repository, follow call chains, compare related implementations, run lightweight checks, and only then commit to a decision. Figure 2 illustrates this failure mode using an issue `sympy-13877`. The candidate PR adds a NaN guard that suppresses a crash, but the function still returns an incorrect nan. Single-turn reviewers approve the symptom fix. Agentic reviewers trace execution upstream, find that `cancel(ret)` is called without assigning its result, and reject the PR with the one-line upstream fix.

Beyond the case study, we compare these two review paradigms quantitatively on our SWE-Review-Bench. Holding the reviewer model fixed to Claude Opus 4.6 [4], agentic review consistently out-

performs single-turn review (diff-only and diff + context settings) on PRs generated by GLM-5, Qwen3-Coder-30B-A3B, and Qwen3-30B-A3B (see Figure 3). The gains appear in both DA and RRR, indicating that agentic review not only makes more reliable approve/request-changes decisions, but also produces diagnoses that lead to stronger downstream revisions. Within the single-turn family, adding retrieved context generally improves over diff-only review, which confirms that extra repository evidence is useful. However, the remaining gap to full agentic review suggests that the main advantage is not merely having more context, but being able to adaptively gather the correct evidence for the specific PR under review. The benefit is also largest on PRs from weaker PR generators: for Qwen3-30B-A3B, agentic review raises RRR from 44.1% under the best single-turn setting to 52.6%, and from the 27.5% no-review baseline to 52.6% overall, nearly doubling the final resolve rate. This pattern suggests that interactive evidence collection is especially valuable when candidate PRs are lower quality and more likely to contain partial or non-local fixes.

This pattern becomes even clearer when we stratify the 1,384 instances in SWE-Review-Bench by difficulty. Specifically, we partition them into easy, medium, and hard tertiles using a post-hoc stratification criterion: the candidate patch’s file-level divergence from the golden patch, and the issue’s number of fail-to-pass tests, which approximates the breadth of behavior that must be validated. The resulting trend is clear: the gap between agentic and single-turn review widens monotonically from easy to hard patches (see Figure 4). This is consistent with our hypothesis that repository exploration matters most when patch correctness cannot be reliably assessed from the diff alone. Appendix B provides extended analyses of agentic reviewer exploration behavior, failure modes, and token consumption.

4 Learning from Agentic Review Trajectories

The previous section establishes agentic review as a stronger inference-time reviewer because it adaptively gathers the repository evidence needed to judge a patch and produce useful feedback. We next examine the broader value of agentic review, e.g., during training or in test-time scaling settings. To support these experiments, we construct and release the SWE-Review-Traj dataset, which records the exploration traces, evidence, decisions, and diagnoses produced by agentic reviewers. This dataset also helps fill a gap in the open-source research community, where high-quality trajectories for agentic code review remain scarce.

4.1 Trajectory Curation and Validation

We source issues from SWE-rebench [6], a large-scale collection of real-world software issues with executable test suites. After filtering to instances with verified solutions and excluding any instance from a repository that appears in SWE-Review-Bench to prevent leakage, we obtain approximately 6k issues. We then generate candidate PRs with the same three models used to construct SWE-Review-Bench (GLM-5, Qwen3-Coder-30B-A3B, and Qwen3-30B-A3B). After removing PRs with empty or oversized patches (>50 KB, which typically indicate large-scale file deletion rather than a targeted fix), we retain 14,156 candidate PRs. For reproducibility, Appendix E lists the full verbatim prompts used for patch generation and review trajectory construction.

To collect complete and transparent review trajectories, we use open-weight GLM-5 with thinking enabled as the teacher, rather than a closed model such as Claude Opus 4.6. The teacher is deployed with the OpenHands-SDK scaffold [37]. To avoid biasing the review toward the candidate PR’s proposed fix, the prompt asks the teacher to first understand the issue and trace the root cause before inspecting the PR (see Appendix E for the full prompt). For each retained candidate PR, the teacher performs one complete code review, and we record the resulting interaction as a training trajectory, yielding 14,156 trajectories before filtering.

Raw teacher trajectories require filtering before they can serve as supervised training data. We first apply decision correctness based on executable patch outcomes, retaining trajectories where the reviewer correctly approves a resolving patch or correctly requests changes on a non-resolving one. This filtering leaves 8,914 trajectories; unless otherwise stated, this decision-correct set is our default training data.

A correct decision does not guarantee a correct diagnosis: a reviewer may request changes on a non-resolving patch while identifying the wrong defect. We therefore audit diagnosis quality with

Table 1: Validation of diagnosis quality.

(a) Semantic validation			(b) Functional validation	
Metric	Opus 4.6	GPT 5.4	Review Completeness	RRR (%)
Diagnosis accuracy	3.15	3.06	No review	3.0
Suggestion correctness	2.79	2.81	Decision only	8.0
Grounding quality	3.54	3.41	Teacher review	21.0
Overall score	3.06	3.09	Oracle review	32.0

two complementary checks: semantic validation asks whether the diagnosis is consistent with the known solution, while functional validation asks whether the text actually helps another agent revise the patch.

Semantic validation. For each trajectory that correctly requests changes, we audit whether the diagnosis text accurately identifies the underlying defect. We compare the diagnosis with the golden patch, and two proprietary judges (Claude Opus 4.6 [4] and GPT-5.4 [24]) rate three dimensions on a 1–5 scale: factual accuracy, correctness of the suggested fix, and grounding in repository evidence. As shown in Table 1a, the mean scores exceed 3.0 on a 5-point scale for both judges, indicating that the diagnoses contain meaningful valid information even though they are not perfect. The agreement between the two judges is substantial (Cohen’s $\kappa = 0.72$), and only 3.3% of paired ratings differ by ≥ 2 points. We also tried stricter semantic filtering by requiring both judges to assign a score of at least 3, which yields a decision-correct and semantic-correct subset of 6,789 trajectories, but this additional filtering has no clear benefit over simply decision-correct filtering on the trained reviewer. Next, we check whether that information is actionable during revision.

Functional validation. Following the RRR protocol in Section 3.1, we sample 100 instances where the teacher correctly requested changes and ask agents to revise each patch given review feedback of varying completeness: (a) no review, (b) decision only, where the patch is marked request-changes but no diagnosis is provided, (c) teacher review, and (d) oracle review, produced by Claude Opus 4.6 with access to the golden patch and the issue’s executable tests. Thus, the oracle review serves as a quality upper bound rather than a deployable reviewer. Conditions (a) and (b) isolate the value of the request-changes signal, while setting (c) measures the additional value of the diagnosis. Table 1b reports RRR under each condition. The results show that the diagnosis carries actionable information beyond the binary decision. A request-changes signal alone improves revision from 3% to 8%, but adding the teacher diagnosis substantially raises the resolve rate to 21%. This recovers about two thirds of the gap between no review and the oracle upper bound (32%).

4.2 Distilling Agentic Review Capabilities

SFT with review trajectories. Having constructed the trajectory corpus, we test whether agentic review behavior can be distilled into smaller open models. We train Qwen3-8B and Qwen3-30B-A3B via SFT on our SWE-Review-Traj dataset. Training details are in Appendix C.

Table 2 shows that SFT with our SWE-Review-Traj dataset substantially improves the base models’ ability to produce parseable review reports, while also improving their decision accuracy. For Qwen3-8B, completion rate rises from approximately 4% to 71–84%, while decision accuracy improves by 18–21 points across the three PR-generator splits. These decision gains translate into higher final resolve rates on the two Qwen splits (50.9→52.8 and 27.5→35.1). Qwen3-30B-A3B shows a similar downstream benefit after SFT, improving RRR on all three splits over its base model. Although SWE-Review-8B provides no RRR gain over no review when reviewing GLM-5 patches, this outcome is expected because the distilled reviewers are much smaller and less capable than GLM-5 itself. Clear gains on GLM-5 patches emerge only when the reviewer is strong enough, as with Claude Opus 4.6.

SFT with mixed issue-resolution and review trajectories. Reviewer distillation shows that a model can learn to judge PRs. We next ask whether the same review trajectories can be mixed with issue-resolution data to produce a single model that is better at both roles: writing patches/PRs

Table 2: **Full evaluation results across reviewers and PR-generator splits.** All values are percentages; green/red subscripts on RRR $\uparrow$ report percentage-point changes relative to the corresponding no-review baseline. The no-review RRR equals the original resolve rate of each PR generator, since all patches are kept unchanged without review.

Agentic Reviewer	GLM-5			Qwen3-Coder-30B-A3B			Qwen3-30B-A3B		
	CR $\uparrow$	DA $\uparrow$	RRR $\uparrow$	CR $\uparrow$	DA $\uparrow$	RRR $\uparrow$	CR $\uparrow$	DA $\uparrow$	RRR $\uparrow$
<i>Baselines</i>									
No Review	—	—	72.2	—	—	50.9	—	—	27.5
<i>Frontier Models</i>									
Claude Opus 4.6	100	75.6	75.2 _{+3.0}	100	80.5	67.3 _{+16.4}	100	89.4	52.6 _{+25.1}
GLM-5	98.9	69.9	75.0 _{+2.8}	97.3	75.7	60.0 _{+9.1}	97.4	86.3	47.5 _{+20.0}
<i>Our Trained Reviewers</i>									
Qwen3-8B (base)	4.1	49.0	72.2 ± 0.0	3.6	49.1	50.9 ± 0.0	3.7	50.8	27.5 ± 0.0
SWE-Review-8B	84.2	68.7	71.6 _{-0.6}	81.4	66.9	52.8 _{+1.9}	71.1	71.6	35.1 _{+7.6}
Qwen3-30B-A3B (base)	84.2	63.2	64.9 _{-7.3}	76.0	54.5	49.1 _{-1.8}	76.3	45.9	28.2 _{+0.7}
SWE-Review-30B-A3B	82.0	69.0	72.6 _{+0.4}	83.1	70.5	53.7 _{+2.8}	87.2	76.5	35.8 _{+8.3}

Table 3: **Adding review trajectories improves code generation at every scale.** Qwen3-8B fine-tuned with issue-resolution trajectories alone vs. paired with an equal volume of review trajectories.

Training data	RR	CR	DA	RRR
Issue-resolution 1k	27.6	9.4	50.0	27.6
Issue-resolution 1k + Review 1k	28.4	67.6	67.4	34.6
Issue-resolution 2k	31.2	13.4	51.7	31.2
Issue-resolution 2k + Review 2k	36.8	85.4	69.5	41.8
Issue-resolution 3k	34.0	33.5	51.6	34.0
Issue-resolution 3k + Review 3k	37.8	87.4	72.3	41.2

and reviewing them. A positive result would suggest that review and repair share a transferable repository-reasoning skill, rather than being two isolated capabilities [2, 19, 39]. Moreover, if the mixed model remains an effective reviewer, it can serve as both the patch/PR generator and the reviewer in a self-contained review-then-revise loop.

To test this, we fine-tune Qwen3-8B under two settings at multiple data scales: an issue-resolution-only baseline trained exclusively on issue-resolution trajectories, and a mixed setting that pairs the same volume of issue-resolution trajectories with an equal volume of review trajectories. We report direct SWE-bench Verified resolve rate (RR), together with CR, DA, and RRR from running the same models in the review-then-revise setting.

We have two main findings. First, mixed training improves the model even when it is used simply as an issue-resolution model: compared with issue-resolution-only SFT, RR rises from 27.6% to 28.4% at 1k data scale, from 31.2% to 36.8% at 2k, and from 34.0% to 37.8% at 3k. Therefore, review trajectories provide useful signal for issue resolution rather than merely teaching a separate judging skill. Second, the mixed model also becomes a usable reviewer. Its CR jumps from 9.4–33.5% under issue-resolution-only training to 67.6–87.4% with mixed training, while DA reaches 67.4–72.3%. This enables a self-contained review-then-revise loop in which the same model generates a patch, reviews it, and revises it, improving final issue resolution rate from 27.6% to 34.6%, from 31.2% to 41.8%, and from 34.0% to 41.2% across the three data scales.

4.3 Agentic Review Enables Effective and Efficient Test-Time Scaling

Test-time scaling for issue resolution is typically implemented by sampling multiple candidate patches and scoring them with a verifier. Existing trained scorers either decode a verdict [12, 26, 35] or attach a scalar head [38], primarily serving to rank candidates and select the best one. Agentic review exposes a richer interface: the approval decision can gate sampling, while the structured diagnosis can be fed back to the issue-resolution model for targeted revision [18].

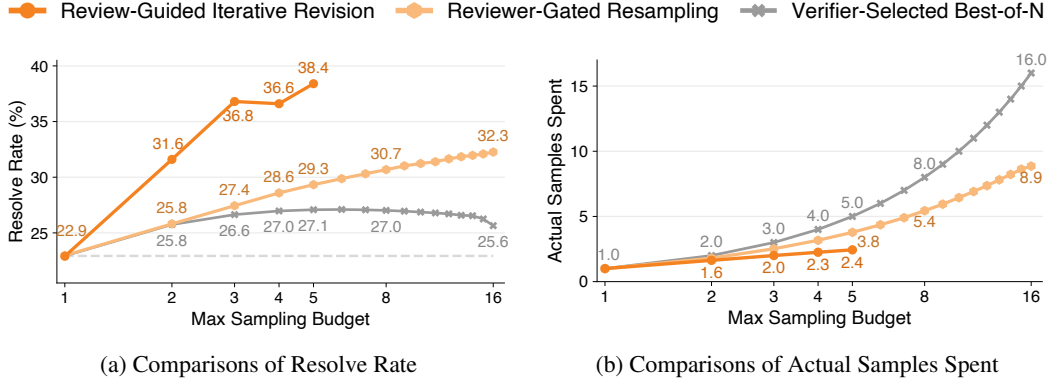


Figure 5: **Agentic review enables effective and efficient test-time scaling for issue resolution.** All methods use Qwen3-30B-A3B as the issue-resolution model and compare size-matched 8B reviewer/verifier judges; both panels use a log-scaled sample-budget x-axis. Review-guided iterative revision achieves the best resolve rate while using far fewer samples on average than independent resampling or verifier-selected best-of- N .

Figure 5 compares three test-time scaling strategies, all using Qwen3-30B-A3B as the issue-resolution model and size-matched 8B models as judges. As a strong published verifier baseline, verifier-selected best-of- N uses SWE-Lego-Verifier-8B [35] to score a fixed set of independent candidates, so its sample cost grows directly with the budget. Reviewer-gated resampling also draws independent candidates, but stops as soon as our trained SWE-Review-8B approves one. Review-guided iterative revision instead turns each rejection into feedback for the next attempt, allowing later samples to revise the current PR rather than restart from scratch.

The integrated curves show that the structured review signal matters more than using approval as a gate alone. From a baseline of 22.9%, review-guided iterative revision reaches 38.4% resolve rate within a maximum budget of five samples, while using only 2.44 samples on average because the loop terminates once the reviewer approves. Reviewer-gated resampling benefits from early stopping but lacks diagnostic feedback: even with a budget of 16 independent attempts, it reaches 32.3% resolve rate and consumes 8.9 samples on average. Verifier-selected best-of- N has no early-exit mechanism, so every increase in budget incurs the full sampling cost. Together, these results show that agentic review enables both effective and efficient test-time scaling for issue resolution. Appendix A provides results with additional coding agents and selection methods, with consistent conclusions.

5 Conclusion

We introduced **SWE-Review**, a framework for closing the loop on AI-assisted issue resolution with agentic code review. Rather than treating review as a one-shot comment-generation task, SWE-Review asks a reviewer agent to inspect the repository, issue, and candidate PR, make an approve/request-changes decision, and produce a structured diagnosis that can guide revision. To evaluate this setting, we constructed **SWE-Review-Bench**, a benchmark of 1,384 AI-generated PRs from 500 SWE-bench Verified issues, and **SWE-Review-Traj**, a corpus of 8,914 decision-correct agentic review trajectories for training open reviewers. Across three PR-generator distributions, agentic review improves both decision accuracy and post-review revision outcomes over fixed-context single-turn review, with the largest gains on harder, more non-local patches. The same trajectories also support effective reviewer distillation, improve issue-resolution training when mixed with patch-generation data, and enable efficient test-time scaling through review-guided iterative revision. Together, these results show that code review is not merely a downstream quality gate for AI-generated PRs, but a central mechanism for turning one-shot patch generation into a closed generate-review-revise process.

Limitations. This work focuses on AI-generated PRs for SWE-style issue resolution, leaving broader review scenarios such as feature implementation, refactoring, documentation, migration, and architectural changes outside our scope. Our metrics are also functional: DA and RRR measure whether a patch resolves the target issue and whether feedback improves revision, but not style, readability, maintainability, security, performance, documentation quality, or project-specific conventions. Finally, our relative comparisons are limited to the selected executable tasks, repositories, model families, prompts, and OpenHands-SDK scaffold. Future work should extend agentic review to broader PR types and non-functional review criteria.

References

- [1] Zhipu AI. Glm-5: from vibe coding to agentic engineering. [arXiv preprint arXiv:2602.15763](https://arxiv.org/abs/2602.15763), 2026. URL <https://arxiv.org/abs/2602.15763>.
- [2] Shengnan An, Zexiong Ma, Zeqi Lin, Nanning Zheng, Jian-Guang Lou, and Weizhu Chen. Learning from mistakes makes llm better reasoner. [arXiv preprint arXiv:2310.20689](https://arxiv.org/abs/2310.20689), 2023.
- [3] Anthropic. Claude code by anthropic. <https://www.anthropic.com/product/claude-code>, 2026.
- [4] Anthropic. Introducing claude opus 4.6. <https://www.anthropic.com/news/claude-opus-4-6>, 2026.
- [5] Anysphere, Inc. Cursor: The Best Way to Code with AI. <https://cursor.com/>, 2026. Accessed: 2026-05-07.
- [6] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. Swe-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. [arXiv preprint arXiv:2505.20411](https://arxiv.org/abs/2505.20411), 2025.
- [7] Cognition AI. Introducing devin, the first ai software engineer. <https://cognition.ai/blog/introducing-devin>, 2024.
- [8] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, et al. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks? [arXiv preprint arXiv:2509.16941](https://arxiv.org/abs/2509.16941), 2025.
- [9] Alexander Frömmgen, Jacob Austin, Peter Choy, Nimesh Ghelani, Lera Kharatyan, Gabriela Surita, Elena Khrapko, Pascal Lamblin, Pierre-Antoine Manzagol, Marcus Revaj, et al. Resolving code review comments with machine learning. In *Proceedings of the 46th international conference on software engineering: software engineering in practice*, pages 204–215, 2024.
- [10] Paul Gauthier. Aider: Ai pair programming in your terminal. <https://github.com/paul-gauthier/aider>, 2023. GitHub repository.
- [11] Hanyang Guo, Xunjin Zheng, Zihan Liao, Hang Yu, Peng Di, Ziyin Zhang, and Hong-Ning Dai. Codefuse-cr-bench: A comprehensiveness-aware benchmark for end-to-end code review evaluation in python projects. [arXiv preprint arXiv:2509.14856](https://arxiv.org/abs/2509.14856), 2025.
- [12] Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. R2e-gym: Procedural environments and hybrid verifiers for scaling open-weights swe agents. [arXiv preprint arXiv:2504.07164](https://arxiv.org/abs/2504.07164), 2025.
- [13] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? [arXiv preprint arXiv:2310.06770](https://arxiv.org/abs/2310.06770), 2023.
- [14] Heng-Yi Li, Shu-Ting Shi, Ferdian Thung, Xuan Huo, Bowen Xu, Ming Li, and David Lo. Deepreview: automatic code review using deep multi-instance learning. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, pages 318–330. Springer, 2019.

- [15] Lingwei Li, Li Yang, Huaxi Jiang, Jun Yan, Tiejian Luo, Zihan Hua, Geng Liang, and Chun Zuo. Auger: automatically generating review comments with pre-training models. In Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pages 1009–1021, 2022.
- [16] Zhiyu Li, Shuai Lu, Daya Guo, Nan Duan, Shailesh Jannu, Grant Jenks, Deep Majumder, Jared Green, Alexey Svyatkovskiy, Shengyu Fu, et al. Codereviewer: Pre-training for automating code review activities. arXiv preprint arXiv:2203.09095, 2022.
- [17] Junyi Lu, Lei Yu, Xiaojia Li, Li Yang, and Chun Zuo. Llama-reviewer: Advancing code review automation with large language models through parameter-efficient fine-tuning. In 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE), pages 647–658. IEEE, 2023.
- [18] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. Advances in neural information processing systems, 36:46534–46594, 2023.
- [19] Dakota Mahan, Duy Van Phung, Rafael Rafailov, Chase Blagden, Nathan Lile, Louis Castri-cato, Jan-Philipp Fränken, Chelsea Finn, and Alon Albalak. Generative reward models. arXiv preprint arXiv:2410.12832, 2024.
- [20] Atharva Naik, Marcus Alenius, Daniel Fried, and Carolyn Rose. Crscore: Grounding automated evaluation of code review comments in code claims and smells. In Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), pages 9049–9076, 2025.
- [21] Doriane Olewicki, Leuson Da Silva, Suhaib Mujahid, Arezou Amini, Benjamin Mah, Marco Castelluccio, Sarra Habchi, Foutse Khomh, and Bram Adams. Impact of llm-based review comment generation in practice: A mixed open-/closed-source user study. arXiv preprint arXiv:2411.07091, 2024.
- [22] OpenAI. Introducing swe-bench verified. <https://openai.com/index/introducing-swe-bench-verified/>, August 2024. Blog post.
- [23] OpenAI. Introducing codex. <https://openai.com/zh-Hans-CN/index/introducing-codex/>, 2026.
- [24] OpenAI. Introducing gpt-5.4. <https://openai.com/zh-Hans-CN/index/introducing-gpt-5-4/>, 2026.
- [25] OpenCode. Opencode. <https://github.com/opencode-ai/opencode>, 2025.
- [26] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with swe-gym. arXiv preprint arXiv:2412.21139, 2024.
- [27] Kristen Pereira, Neelabh Sinha, Rajat Ghosh, and Debojyoti Dutta. Cr-bench: Evaluating the real-world utility of ai code review agents, 2026.
- [28] Qodo. Pr-agent: Ai-powered pull request analysis and feedback. <https://github.com/The-PR-Agent/pr-agent>, 2023. GitHub repository.
- [29] Qwen. Qwen3-coder: Agentic coding in the world. <https://qwenlm.github.io/blog/qwen3-coder/>, 2025.
- [30] Shu-Ting Shi, Ming Li, David Lo, Ferdian Thung, and Xuan Huo. Automatic code review by learning the revision of source code, 2019.

- [31] KaShun Shum, Binyuan Hui, Jiawei Chen, Lei Zhang, Jiaxi Yang, Yuzhen Huang, Junyang Lin, Junxian He, et al. Swe-rm: Execution-free feedback for software engineering agents. arXiv preprint arXiv:2512.21919, 2025.
- [32] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. arXiv preprint arXiv:2408.03314, 2024.
- [33] Tao Sun, Jian Xu, Yuanpeng Li, Zhao Yan, Ge Zhang, Lintao Xie, Lu Geng, Zheng Wang, Yueyan Chen, Qin Lin, et al. Bitsai-cr: Automated code review via llm in practice. In Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, pages 274–285, 2025.
- [34] Xunzhu Tang, Kisub Kim, Yewei Song, Cedric Lothritz, Bei Li, Saad Ezzini, Haoye Tian, Jacques Klein, and Tegawendé F Bissyandé. Codeagent: Autonomous communicative agents for code review. In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pages 11279–11313, 2024.
- [35] Chaofan Tao, Jierun Chen, Yuxin Jiang, Kaiqi Kou, Shaowei Wang, Ruoyu Wang, Xiaohui Li, Sidi Yang, Yiming Du, Jianbo Dai, et al. Swe-lego: Pushing the limits of supervised fine-tuning for software issue resolving. arXiv preprint arXiv:2601.01426, 2026.
- [36] Rosalia Tufano, Luca Pascarella, Michele Tufano, Denys Poshyvanyk, and Gabriele Bavota. Towards automating code review activities, 2021.
- [37] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. arXiv preprint arXiv:2407.16741, 2024.
- [38] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Learning to verify ai-generated code. <https://openhands.dev/blog/20260305-learning-to-verify-ai-generated-code>, March 2026. Blog post.
- [39] Yubo Wang, Xiang Yue, and Wenhui Chen. Critique fine-tuning: Learning to critique is more effective than learning to imitate. arXiv preprint arXiv:2501.17703, 2025.
- [40] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. arXiv preprint arXiv:2505.09388, 2025.
- [41] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. Advances in Neural Information Processing Systems, 37:50528–50652, 2024.
- [42] John Yang, Kilian Lieret, Carlos E Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. Swe-smith: Scaling data for software engineering agents. arXiv preprint arXiv:2504.21798, 2025.
- [43] Lei Zhang, Yongda Yu, Minghui Yu, Xinxin Guo, Zhengqi Zhuang, Guoping Rong, Dong Shao, Haifeng Shen, Hongyu Kuang, Zhengfeng Li, et al. Aacr-bench: Evaluating automatic code review with holistic repository-level context. arXiv preprint arXiv:2601.19494, 2026.
- [44] Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, Elsie Nallipogu, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Dongmei Zhang. Swe-bench goes live! arXiv preprint arXiv:2505.23419, 2025.
- [45] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. Autocoderover: Autonomous program improvement. In Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, pages 1592–1604, 2024.

- [46] Yuntong Zhang, Zhiyuan Pan, Imam Nur Bani Yusuf, Haifeng Ruan, Ridwan Shariffdeen, and Abhik Roychoudhury. Code review agent benchmark, 2026.
- [47] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623, 2023.

A Additional Test-Time Scaling Results

Section 4.3 focuses on three test-time scaling strategies when the issue-resolution model is Qwen3-30B-A3B and the judges are size-matched 8B models. This appendix extends those results in two directions: a single-model iterative loop (Section A.1) and a broader comparison of selection rules across two coding agents (Section A.2).

A.1 Single-Model Iterative Review-Revision

The main-text comparisons use separate models for generation and review (Qwen3-30B-A3B or Qwen3-Coder-30B-A3B as the issue-resolution agent, SWE-Review-8B as the judge). Here we ask whether a single checkpoint can play both roles in a self-contained generate-review-revise loop.

Setup. We use Qwen3-8B fine-tuned with mixed issue-resolution and review trajectories at the 3k+3k scale (the “Issue-resolution 3k + Review 3k” row in Table 3). The same model generates the initial patch, reviews it, and revises based on its own structured feedback when it decides to request changes. The loop runs for up to 5 samples (1 initial generation + 4 revision rounds) or until the model approves its own patch.

Results. Table 4 reports resolve rates on SWE-bench Verified ($n=500$). RR increases steadily from 34.8% (sample 1, no review) to 44.0% (sample 5). Cumulative oracle coverage, crediting any instance where at least one sample produces a resolving patch, reaches 47.2% (236/500) after 5 samples. These results confirm that mixed training equips a single checkpoint with sufficient capability to serve as both generator and reviewer, enabling steady gains through iterative self-review.

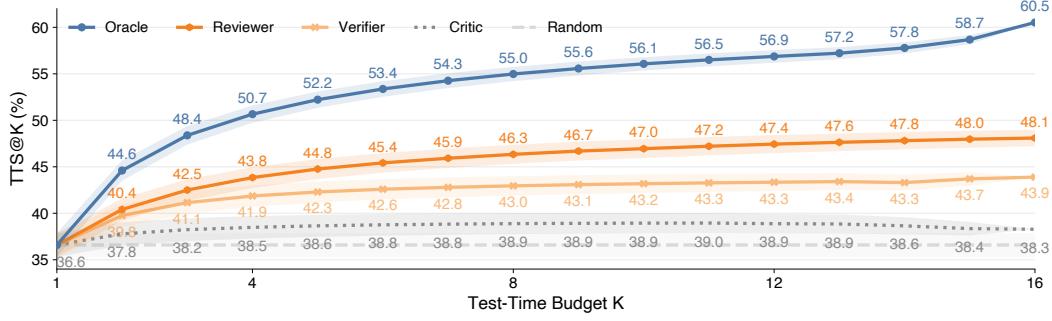
Table 4: **A single mixed-trained model supports effective review-guided iterative revision.** Qwen3-8B (mixed 3k+3k SFT from Table 3) serves as both the issue-resolution agent and the reviewer on SWE-bench Verified ($n=500$), up to 5 samples. RR is the resolve rate of the reviewer-approved patch at each sample; Cumulative Oracle RR credits any instance where at least one sample produces a resolving patch.

Sample	RR (%)	Cumulative Oracle RR (%)
1	34.8	34.8
2	40.6	44.4
3	43.0	46.4
4	43.4	46.8
5	44.0	47.2

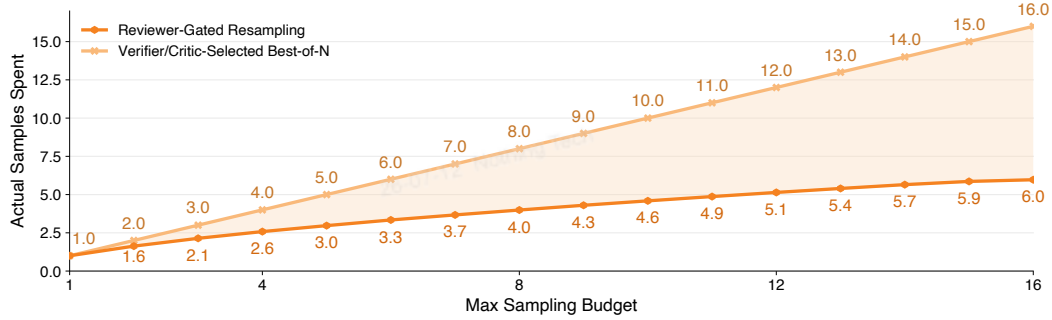
A.2 Comparisons Across More Coding Agents and Selection Methods

Section 4.3 compares three test-time scaling strategies with a single coding agent. Here we extend the comparison to two coding agents (Qwen3-30B-A3B and Qwen3-Coder-30B-A3B) and add OpenHands Critic as an additional selection baseline. In all cases the reviewer is our distilled SWE-Review-8B model.

Protocols and baselines. For a maximum test-time budget $K \in \{1, \dots, 16\}$, we draw up to K independent issue-resolution trajectories (same scaffold and evaluation as Section 4.3). We compare five selection rules, named to match the figure legends:



(a) TTS@K across selectors.



(b) Actual samples spent vs. maximum budget.

Figure 6: **Test-time scaling with Qwen3-30B-A3B as the coding agent.** Reviewer dominates Verifier and Critic across all budgets, while using far fewer samples due to early stopping.

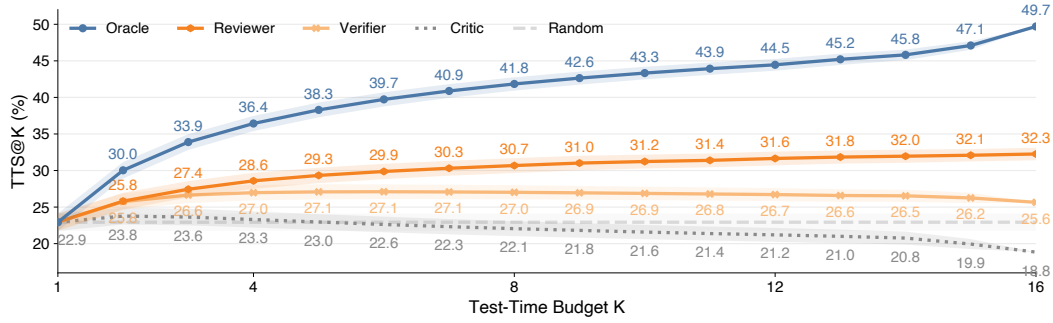
1. **Oracle**: reports $\text{Pass}@K$, i.e., the fraction of instances for which at least one of the K samples resolves the issue, serving as an upper bound on any selector.
2. **Reviewer**: our trained SWE-Review-8B used in a reviewer-gated resampling protocol where candidates are drawn sequentially and the loop stops as soon as the reviewer approves a patch, or when K attempts are exhausted.
3. **Verifier**: the published SWE-Lego-Verifier-8B model [35], which scores all K candidates and returns the highest-scoring patch (best-of- K).
4. **Critic**: OpenHands Critic [38], another trained scalar scorer used in the same best-of- K fashion.
5. **Random**: chooses one of the K samples uniformly at random, serving as a no-selection baseline.

We report $\text{TTS}@K$, defined as the SWE-bench Verified resolve rate achieved by each selection rule at budget K .

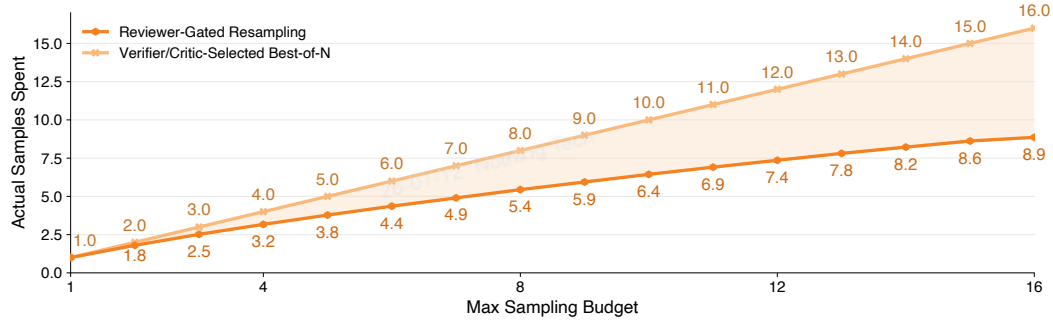
Results. In all figures, shaded bands indicate the standard deviation over random orderings of the K candidate samples drawn for each instance. The second panel of each figure shows the average number of samples actually consumed under reviewer-gated resampling versus the identity line $y=K$ for fixed best-of- K methods.

Figure 6 reports results with Qwen3-30B-A3B as the coding agent. Reviewer-gated resampling improves monotonically with K and clearly dominates both Verifier and Critic at every budget, while the Critic barely outperforms Random. At $K=16$, Reviewer reaches 32.3% resolve rate versus 25.6% for Verifier and 18.8% for Critic. On the efficiency side, Reviewer uses only 8.9 samples on average at $K=16$ (a 44.4% reduction over the fixed budget of 16).

Figure 7 repeats the comparison with Qwen3-Coder-30B-A3B. Absolute resolve rates are higher due to the stronger coding agent, but the relative ordering is unchanged: Reviewer remains the best practical selector. At $K=16$, Reviewer reaches 48.1% versus 44.4% for Verifier, while using only 6.0 samples on average (a 62.5% reduction). Notably, the Critic underperforms Random at larger K , suggesting that its scalar scores can misrank candidates from this agent.



(a) TTS@K across selectors.



(b) Actual samples spent vs. maximum budget.

Figure 7: **Test-time scaling with Qwen3-Coder-30B-A3B as the coding agent.** At $K=16$, Reviewer reaches 48.1% vs. 44.4% for Verifier while using only 6.0 samples on average (62.5% reduction).

B Extended Analysis of Reviewer Behavior

This section provides extended analyses of the agentic review pipeline, examining reviewer exploration behavior (Section B.1), failure modes (Section B.2), and token consumption (Section B.3).

B.1 Reviewer Exploration Behavior

We analyze the exploration trajectories of three agentic reviewers (Claude Opus 4.6, SWE-Review-30B-A3B, and SWE-Review-8B) to characterize how exploration depth and efficiency relate to review quality.

Cross-model comparison. Table 5 compares aggregate exploration behavior across the three reviewers. The central finding is that exploration efficiency, rather than volume, predicts review quality. Opus 4.6 achieves the highest DA (81.8% weighted average) while using the fewest interaction steps (24.1 on average), the fewest files read (1.3), and the fewest total tokens (148K). In contrast, SWE-Review-8B uses $2.4\times$ more steps (57.3) and $16\times$ more tokens (2.36M) yet achieves only 69.1% DA.

Reproducer execution rate is the strongest single cross-model predictor of DA: Opus runs reproducers in 99.9% of trials compared to 76.7% for SWE-Review-8B, suggesting that the ability to reliably formulate and execute verification tests is a key competency gap that distillation has not yet fully closed. The distilled models exhibit a compensatory exploration pattern: they read $3.5\text{--}3.8\times$ more files and perform $2.6\text{--}3.3\times$ more grep searches than Opus, yet achieve 10–13 percentage points lower DA. This indicates that the smaller models lack the capacity to efficiently identify decision-relevant information and instead resort to broad, undirected search. Opus confines itself to the diff region in 63% of instances while still running reproducers in nearly all cases, implying that its superior accuracy stems from a targeted strategy of executing the reproduction script and reasoning over the result rather than from exhaustive codebase traversal.

Table 5: **Exploration behavior by reviewer model.** Opus 4.6 achieves the highest DA (81.8%) while consuming an order of magnitude fewer tokens (148K vs. 2.09–2.36M) and fewer than half the steps of the distilled models, indicating that review quality is driven by exploration precision rather than volume.

Metric	Opus 4.6	SWE-Review-30B-A3B	SWE-Review-8B
DA (Table 2)	81.8%	72.0%	69.1%
CR (Table 2)	100%	84.1%	78.9%
Avg steps	24.1	53.6	57.3
Avg files read	1.3	4.6	5.0
Avg code executions	17.8	32.7	35.1
% explored beyond diff	37.0%	89.2%	92.4%
% ran reproducer	99.9%	83.6%	76.7%
Avg total tokens	148K	2.09M	2.36M
Avg grep searches	3.3	8.6	10.9

DA-correct versus DA-incorrect exploration. Table 6 compares behavioral metrics between correctly and incorrectly decided instances within each model. All three reviewers exhibit the same pattern: DA-incorrect cases consume significantly more steps, code executions, and tokens (all $p < 0.001$ by Mann–Whitney U test). Incorrect reviews by Opus 4.6 consume 33% more tokens (186K vs. 140K), while incorrect reviews by SWE-Review-8B consume 42% more tokens (2.91M vs. 2.05M). This suggests that when a model fails to reach a confident judgment early in its trajectory, additional tool calls do not rescue the decision but rather reflect unproductive deliberation on inherently harder instances.

The SWE-Review-30B-A3B results reveal a particularly informative asymmetry: DA-incorrect cases exhibit a higher reproducer rate (93.0%) than DA-correct cases (80.8%, $p < 0.05$). This suggests that for this model, reproducer output can actively mislead the reasoning process rather than anchor it, perhaps because the model lacks the capacity to correctly interpret ambiguous test outcomes.

Table 6: **Exploration behavior on DA-correct versus DA-incorrect instances.** Across all three models, incorrect decisions consume significantly more computation ($p < 0.001$), indicating that excessive exploration is a symptom of uncertainty rather than a path to accuracy.

Model	Metric	DA Correct	DA Incorrect	p
Opus 4.6	Avg steps	23.2	28.1	$< 0.001^{***}$
	Avg code exec	17.1	20.9	$< 0.001^{***}$
	Avg tokens	140K	186K	$< 0.001^{***}$
	% ran reproducer	100.0%	99.6%	ns
SWE-Review-8B	Avg steps	53.3	64.5	$< 0.001^{***}$
	Avg code exec	32.3	40.0	$< 0.001^{***}$
	Avg tokens	2.05M	2.91M	$< 0.001^{***}$
	% ran reproducer	78.3%	73.7%	ns
SWE-Review-30B-A3B	Avg steps	51.9	59.2	$< 0.001^{***}$
	Avg code exec	31.5	36.8	$< 0.001^{***}$
	Avg tokens	1.98M	2.47M	$< 0.001^{***}$
	% ran reproducer	80.8%	93.0%	$< 0.05^*$

Difficulty stratification. Table 7 stratifies Opus 4.6 exploration metrics by difficulty tertile. All exploration indicators increase monotonically with difficulty (steps +74%, files read +111%, beyond-diff rate +200% from easy to hard), while DA decreases by 9.9 percentage points. Crucially, this adaptive allocation differs from the unproductive exploration observed for incorrect predictions in Table 6: here, the additional effort on hard instances is a deliberate strategy that sustains DA at 76.8% even on the most challenging tertile. The 10 percentage point DA gap despite a 74% increase in compute expenditure quantifies the diminishing returns of exploration and suggests that the remaining errors on hard instances stem from reasoning limitations rather than insufficient information

gathering. The relationship between step count and accuracy is further quantified by grouping trials into step quartiles: the fastest quartile (≤ 17 steps) achieves 90.8% DA, while the slowest (> 28 steps) achieves only 69.4% (odds ratio 4.32, $p < 0.001$), suggesting that step count can serve as a lightweight confidence proxy for abstention or escalation strategies.

Table 7: **Opus 4.6 exploration by difficulty tertile.** The reviewer adaptively scales exploration with difficulty (18.3 to 31.8 steps, 18.4% to 55.1% beyond-diff rate), while DA degrades gracefully from 86.7% to 76.8%.

Difficulty	Steps	Files	Code Exec	% Beyond Diff	DA (%)
Easy	18.3	0.9	14.0	18.4%	86.7
Medium	22.7	1.3	16.8	39.7%	82.4
Hard	31.8	1.9	23.0	55.1%	76.8

B.2 Failure Mode Analysis

We analyze the DA errors made by Claude Opus 4.6 across all three PR-generator splits. The reviewer produces 272 total errors: 167 False Approvals (FA, approving an unresolved patch) and 105 False Rejections (FR, rejecting a resolved patch). We annotate root causes using an LLM-as-judge with access to the issue description, candidate patch, golden patch, and reviewer report, successfully categorizing 259 of these errors.

Error distribution by split. Table 8 breaks down FA and FR rates across the three PR-generator splits. The error composition shifts with patch quality: on the high-quality GLM-5 split (72.2% resolve rate), 83% of errors are FA because the remaining 27.8% of incorrect patches are necessarily close to correct, making them harder to distinguish from resolving ones. On the lower-quality Qwen3-30B-A3B split (27.5% resolve rate), FR slightly predominates (57%). This implies that scaling reviewer capability matters most at the frontier: as PR generators improve, the marginal patch that slips through review becomes increasingly subtle, and reviewer error concentrates almost entirely on false approvals.

Table 8: **False Approval and False Rejection rates by PR-generator split.** FA dominates when patch quality is high (20.2% for GLM-5 vs. 4.6% for Qwen3-30B-A3B); overall DA paradoxically improves on weaker generators (89.4% vs. 75.6%) because correct rejections are easier than catching subtle bugs in near-correct patches.

	GLM-5	Qwen3-Coder-30B-A3B	Qwen3-30B-A3B	Total
False Approve	101 (20.2%)	43 (8.6%)	23 (4.6%)	167
False Reject	21 (4.2%)	54 (10.8%)	30 (6.0%)	105
Total Errors	122 (24.4%)	97 (19.5%)	53 (10.6%)	272
DA	75.6%	80.5%	89.4%	81.8%

Root cause breakdown. Table 9 reports the LLM-judge annotation of root causes. Three causes account for nearly all FA errors in roughly equal proportion: subtle logic bugs that survive code inspection (37.7%), insufficient reproducer coverage (32.1%), and cross-file reasoning gaps (29.0%). The “no reproducer” category is empty because Opus runs reproducers in 99.9% of trials (Table 5); FA failures therefore stem from test quality rather than test absence. Among FR errors, code misreading is the dominant cause (44.3%), pointing to failures in basic comprehension rather than reasoning, while the 25.8% attributed to “non-standard but correct” solutions indicates a bias toward canonical implementation patterns. These findings suggest two complementary improvement directions: augmenting the reviewer with stronger test-adequacy analysis to reduce false approvals, and grounding rejection decisions in explicit functional equivalence checks to reduce false rejections.

These results reveal a test paradox: the reviewer’s primary verification tool, test execution, simultaneously drives both failure modes. Among FA review reports, 82% contain the keyword “pass,” indicating that agent-written tests provided false confidence. Furthermore, 62% of FA patches include test files written by the patch-generating AI, which naturally validate the same mental model as the patch itself. Conversely, among FR review reports, 42% contain the keyword “fail,” where

Table 9: **Root cause breakdown for Opus 4.6 review errors** ($n=259$). FA is driven by subtle logic bugs (37.7%) and insufficient reproducers (32.1%), while FR stems primarily from misreading code (44.3%), indicating that the two error types reflect fundamentally different cognitive failures.

Error Type	Root Cause	Count	%
False Approve ($n=162$)	Subtle logic bug	61	37.7%
	Insufficient reproducer	52	32.1%
	Cross-file reasoning gap	47	29.0%
	Other	2	1.2%
False Reject ($n=97$)	Misread code	43	44.3%
	Non-standard but correct	25	25.8%
	Other	24	24.7%
	Misidentified root cause	5	5.2%

the reviewer attributed failures in its own imperfect tests to defects in the patch rather than to flaws in the test logic.

Error distribution by difficulty. Table 10 stratifies errors by difficulty tertile. Nearly half of all false approvals (80) concentrate in the hard tertile, consistent with the root-cause analysis: hard instances are more likely to involve subtle logic bugs and cross-file dependencies that evade surface-level review. The FR rate is non-monotonic, peaking at the medium tertile (9.5%), which suggests that medium-difficulty tasks produce patches employing less obvious solution strategies that trigger the “non-standard but correct” failure mode. The relatively low FR rate on hard instances (6.3%) likely reflects a selection effect: patches that resolve genuinely hard issues tend to be unambiguously substantive, giving the reviewer clearer positive signals. These patterns suggest that difficulty-aware review allocation, where hard instances receive deeper analysis or multi-round verification, could yield the largest marginal gains in overall DA.

Table 10: **Error distribution by difficulty tertile** (Opus 4.6). FA rate more than doubles from easy to hard (7.6% to 16.7%) while FR rate is non-monotonic, confirming that task difficulty disproportionately degrades the reviewer’s ability to catch incorrect patches.

Difficulty	FA	FR	FA Rate	FR Rate	Error Rate
Easy	40	30	7.6%	5.7%	13.3%
Medium	39	45	8.2%	9.5%	17.6%
Hard	80	30	16.7%	6.3%	23.0%

B.3 Token Consumption

We quantify the token consumption of the closed-loop review pipeline. Because the three PR-generator models span a wide capability range (GLM-5: 72.2% resolve rate, Qwen3-Coder-30B-A3B: 50.9%, Qwen3-30B-A3B: 27.5%), the comparison reveals how token budgets shift with patch quality. Claude Opus 4.6 is excluded from cross-model token comparisons because its API-billed counts omit reasoning tokens.

Closed-loop token breakdown. Table 11 decomposes the three stages of the closed-loop pipeline (patch generation, agentic review, and revision) with SWE-Review-8B as the reviewer.

The token budget shifts with generator quality. For the strong GLM-5 split, patch generation, review, and revision each consume roughly a third of the budget. As generator quality decreases, review increasingly dominates (from 36% to 66%). The near-invariance of review tokens across generators of widely differing quality (72.2% vs. 27.5% resolve rate) suggests that the review process has a relatively fixed computational cost driven by the complexity of understanding the issue and code-base, not by the quality of the patch under review. By contrast, revision tokens drop sharply for weaker generators (from 2,199K to 335K), indicating that the reviewer identifies weaker patches as less amenable to incremental repair and terminates revision early.

Table 11: **Per-stage token consumption in the closed-loop pipeline** with SWE-Review-8B as the reviewer. Review token consumption remains nearly constant across generators (2.19–2.46M), while patch generation and revision costs scale with generator capability, indicating that review functions as a fixed-cost understanding phase.

Split	Stage	Mean Tokens	Rounds	Share
GLM-5	Patchgen	2,226K	59.8	32.4%
	Review	2,448K	59.0	35.6%
	Revision	2,199K	66.1	32.0%
Qwen3-Coder-30B-A3B	Patchgen	1,081K	37.3	24.3%
	Review	2,460K	58.7	55.2%
	Revision	913K	35.7	20.5%
Qwen3-30B-A3B	Patchgen	800K	27.9	24.1%
	Review	2,189K	54.8	65.9%
	Revision	335K	15.8	10.1%

Review tokens by difficulty. Table 12 stratifies Claude Opus 4.6 review token consumption by instance difficulty. Although Opus token counts are not comparable to local models (due to excluded reasoning tokens), the within-model comparison across difficulty levels is informative. For strong generators like GLM-5, most easy and medium instances produce correct patches that the reviewer can accept relatively quickly, so hard instances stand out with a 38% token premium (207K vs. 150K). For Qwen3-30B-A3B, patches are frequently incorrect regardless of difficulty, compressing the token range to only 18% (142K vs. 120K): the reviewer reaches a rejection decision at similar cost whether the instance is easy or hard. The round counts corroborate this interpretation, with the gap between easy and hard narrowing from 5.1 rounds (GLM-5) to 2.4 rounds (Qwen3-30B-A3B).

Table 12: **Claude Opus 4.6 review tokens by difficulty and PR-generator split.** The difficulty premium is larger for high-quality generators (38% increase from easy to hard for GLM-5) than for weaker ones (18% for Qwen3-30B-A3B). Token counts exclude reasoning tokens and are used for within-model comparison only.

Split	Difficulty	Mean Tokens	Rounds
GLM-5	Easy	150K	24.5
	Medium	161K	25.0
	Hard	207K	29.6
Qwen3-Coder-30B-A3B	Easy	121K	21.7
	Medium	134K	22.8
	Hard	175K	26.7
Qwen3-30B-A3B	Easy	120K	21.3
	Medium	127K	21.9
	Hard	142K	23.7

Reviewer token efficiency. Table 13 compares the three open-weight agentic reviewers on token consumption and review quality. The efficiency gap between GLM-5 (1,685K tokens per correct decision) and the SWE-Review models (3,190K and 3,423K) arises from a compounding effect: GLM-5 consumes 45% fewer tokens per review on average while simultaneously achieving higher DA (77.3% vs. 72.0% and 69.1%). SWE-Review-8B, despite consuming marginally more tokens than SWE-Review-30B-A3B, achieves lower accuracy, yielding the worst token efficiency. This suggests that simply scaling reviewer inference compute without corresponding gains in reasoning quality leads to diminishing returns on the tokens-per-correct-decision metric.

Test-time scaling token efficiency. Table 14 compares the token efficiency of the three test-time scaling strategies from Section 4.3, all using Qwen3-30B-A3B as the issue-resolution model (baseline resolve rate 22.9%). The verifier achieves only modest gains (25.6%, +2.7pp) despite consuming 16 full samples, because its scalar scoring lacks the semantic depth to reliably distinguish correct from incorrect patches at low base rates. Reviewer-gated resampling improves resolve rate to 32.3% but at extreme cost (26,500K tokens), because it discards and regenerates patches from

Table 13: **Reviewer token consumption and quality.** GLM-5 achieves the lowest cost per correct decision (1,685K tokens), reflecting a 1.9–2.0 $\times$ efficiency advantage from both lower absolute token usage and higher DA. Claude Opus 4.6 is excluded because its API-billed token counts omit reasoning tokens.

Reviewer	Mean Tokens	Rounds	Avg DA	Tokens/correct
GLM-5	1,302K	44.1	77.3%	1,685K
SWE-Review-30B-A3B	2,297K	56.3	72.0%	3,190K
SWE-Review-8B	2,366K	57.5	69.1%	3,423K

scratch rather than building on prior attempts. Review-guided iterative revision achieves the highest resolve rate (38.4%) within a maximum budget of $K=5$ samples while using only 2.44 on average, by converting review feedback into targeted revisions that accumulate progress across iterations. This 6.5 $\times$ token-efficiency gap (2.28 vs. 0.35 pp/Mtok) quantifies the value of directed feedback over independent resampling as a test-time scaling strategy.

Table 14: **Test-time scaling token efficiency.** Review-guided iterative revision achieves 2.28 pp/Mtok, a 6.5 $\times$ efficiency advantage over reviewer-gated resampling (0.35 pp/Mtok). All methods use Qwen3-30B-A3B (baseline RR 22.9%).

Method	Tokens/Instance	Samples	RR (%)	pp/Mtok
Baseline (1 sample)	672K	1	22.9	—
Verifier best-of- N ($N=16$)	11,136K	16	25.6	0.24
Reviewer-gated resampling	26,500K	8.86	32.3	0.35
Review-guided iterative ($K=5$)	6,800K	2.44	38.4	2.28

C SFT Training Details

This appendix provides full reproduction details for all supervised fine-tuning experiments reported in the paper.

C.1 Infrastructure

Hardware. All training runs are conducted on a distributed cluster with multiple high-memory accelerator nodes. Each node contains eight accelerators and supports multi-node distributed training for long-context full-parameter fine-tuning. Larger MoE models are trained on nodes with larger device memory to accommodate their increased memory footprint.

Software. We use Python 3.12, PyTorch 2.6+, transformers 4.55.0, DeepSpeed 0.16+, LLaMA-Factory ($\leq 5.2.0$), and Flash Attention 2.

Training framework. We use LLaMA-Factory as the training frontend with DeepSpeed ZeRO Stage 3 (without CPU offload) for full-parameter distributed training.

C.2 Hyperparameters

Unless otherwise noted for specific ablations, we train with learning rate 10^{-4} , cosine learning-rate scheduler, max gradient norm 1.0, weight decay 0.01, warmup ratio 0.1, and BF16 mixed precision. Per-device batch size is 1 with gradient accumulation steps 8 across eight accelerators, yielding an effective batch size of 64. We apply YaRN RoPE scaling to a cutoff length of 131,072 tokens. The chat template is `qwen3_nothink` (thinking tokens disabled) so that `<think>` blocks do not interfere with tool calling at inference time.

C.3 Training Optimizations

- **Liger Kernel:** Optimized fused kernels for RMSNorm, SwiGLU, and CrossEntropy that reduce memory without changing numerical precision.

- **Unsloth gradient checkpointing:** Fine-grained activation recomputation supporting longer sequences within the same memory budget.
- **Attention implementation:** Flash Attention 2.
- **YaRN RoPE scaling:** Extends the 32K native context of Qwen3 to 131K via frequency-segmented scaling.
- **Turn masking:** Loss is computed only on successful assistant turns; system/user turns and error-preceding assistant turns are masked.

C.4 Training Time and Compute

Wall-clock training time is about 30 hours total for Qwen3-8B with $\sim 9,000$ training examples, and about 70 hours total for Qwen3-30B-A3B with $\sim 9,000$ examples. MoE models activate only a subset of experts per token, so their effective compute is lower than same-parameter dense models. Training time variance is dominated by long-tail samples ($>64K$ tokens).

C.5 Data Preprocessing

Training data follows this pipeline:

1. Raw agent trajectories are cleaned and standardized.
2. `tokenizer.apply_chat_template()` renders each trajectory into the model's native chat format with correct special tokens.
3. Rendered text is split by turn; error-preceding assistant turns are flagged for loss masking.
4. Final output is ShareGPT JSON with per-turn mask annotations.

The tokenizer must exactly match the target base model.

D Case Study: sympy-13877

This appendix expands the case study referenced in Section 3.2 and Figure 2.

Overview. The case compares the same candidate PR under two review settings: single-turn Opus with diff+context and agentic Opus. The candidate PR does not resolve the issue: it fails the hidden `test_determinant` test because the determinant still returns `nan`. The single-turn reviewer nevertheless approves the patch, while the agentic reviewer correctly requests changes and is scored as correct by the evaluation harness. The central failure mode is that the candidate patch suppresses a downstream crash in `factor_terms` but leaves the determinant computation returning `nan`.

D.1 Raw Issue

```
\begin{verbatim}
Matrix determinant raises Invalid NaN comparison with particular symbolic entries
>>> from sympy import *
>>> from sympy.abc import a
>>> f = lambda n: det(Matrix([[i + a*j for i in range(n)] for j in range(n)]))
>>> f(1)
0
>>> f(2)
-a
>>> f(3)
2*a*(a + 2) + 2*a*(2*a + 1) - 3*a*(2*a + 2)
>>> f(4)
0
>>> f(5)
nan
>>> f(6)
Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
    f(6)
  File "<pyshell#2>", line 1, in <lambda>
    f = lambda n: det(Matrix([[i + a*j for i in range(n)] for j in range(n)]))
  File "sympy/matrices/expressions/determinant.py", line 53, in det
    return Determinant(matexpr).doit()
  File "sympy/matrices/expressions/determinant.py", line 37, in doit

```

```

        return self.arg._eval_determinant()
File "sympy/matrices/matrices.py", line 270, in _eval_determinant
    return self.det()
File "sympy/matrices/matrices.py", line 416, in det
    return self._eval_det_bareiss()
File "sympy/matrices/matrices.py", line 213, in _eval_det_bareiss
    return cancel(bareiss(self))
File "sympy/matrices/matrices.py", line 211, in bareiss
    return sign*bareiss(self._new(mat.rows - 1, mat.cols - 1, entry), pivot_val)
File "sympy/matrices/matrices.py", line 211, in bareiss
    return sign*bareiss(self._new(mat.rows - 1, mat.cols - 1, entry), pivot_val)
File "sympy/matrices/matrices.py", line 211, in bareiss
    return sign*bareiss(self._new(mat.rows - 1, mat.cols - 1, entry), pivot_val)
[Previous line repeated 1 more times]
File "sympy/matrices/immutable.py", line 55, in _new
    rows, cols, flat_list = cls._handle_creation_inputs(*args, **kwargs)
File "sympy/matrices/matrices.py", line 2041, in _handle_creation_inputs
    for j in range(cols)]])
File "sympy/matrices/matrices.py", line 2041, in <listcomp>
    for j in range(cols)]])
File "sympy/matrices/matrices.py", line 208, in entry
    cancel(ret)
File "sympy/polys/polytools.py", line 6423, in cancel
    f = factor_terms(f, radical=True)
File "sympy/core/exprtools.py", line 1193, in factor_terms
    return do(expr)
File "sympy/core/exprtools.py", line 1189, in do
    *[do(a) for a in p.args])
File "sympy/core/exprtools.py", line 1189, in <listcomp>
    *[do(a) for a in p.args])
File "sympy/core/exprtools.py", line 1171, in do
    if all(a.as_coeff_Mul()[0] < 0 for a in list_args):
File "sympy/core/exprtools.py", line 1171, in <genexpr>
    if all(a.as_coeff_Mul()[0] < 0 for a in list_args):
File "sympy/core/expr.py", line 323, in __lt__
    raise TypeError("Invalid NaN comparison")
TypeError: Invalid NaN comparison

```

Correct me if I'm wrong but isn't the Bareiss algorithm only valid for integer matrices, which cannot be assumed here?

\end{verbatim}

D.2 Raw Candidate PR

We first present the candidate PR title and body exactly as submitted. Then we show the raw patch. A key detail to keep in mind is that this patch edits `sympy/core/exprtools.py` to guard a comparison, rather than modifying the Bareiss determinant path where the root cause is later identified.

The original PR title is:

```

\begin{verbatim}
fix: prevent NaN comparison error in factor_terms with symbolic matrices
\end{verbatim}

```

The original PR body is:

```

\begin{verbatim}
This fix addresses an issue where the Bareiss algorithm for computing matrix
determinants would crash with 'Invalid NaN comparison' when dealing with
particular symbolic entries in 6x6 matrices.

The problem occurred in 'sympy/core/exprtools.py' in the 'factor_terms' function
at line 1179, where the code attempted to compare a coefficient extracted via
'as_coeff_Mul()[0]' with 0 using '< 0'. In certain symbolic computations, this
coefficient could be NaN, leading to a TypeError.

The fix adds proper validation to check if the coefficient is a valid real finite
number before attempting the comparison. This prevents the crash while preserving
all existing functionality.

```

```

Before this fix:
- f(6) would raise TypeError: Invalid NaN comparison

After this fix:
- f(6) returns nan (consistent with the behavior of f(5))
- All other matrix determinant computations remain unchanged
- No regressions in normal functionality
\end{verbatim}

```

The patch below introduces `_is_neg_coeff` and replaces a direct `< 0` comparison with a guarded check over finite real numbers. This eliminates the crash symptom in one downstream location, but does not by itself establish whether determinant correctness is restored.

The raw patch is:

```

\begin{verbatim}
diff --git a/sympy/core/exprtools.py b/sympy/core/exprtools.py
index 4e0de9ec1..e4a9b7cb6 100644
--- a/sympy/core/exprtools.py
+++ b/sympy/core/exprtools.py
@@ -1176,7 +1176,14 @@ def do(expr):
     if p.is_Add:
         list_args = [do(a) for a in Add.make_args(p)]
         # get a common negative (if there) which gcd_terms does not remove
-    if all(a.as_coeff_Mul()[0] < 0 for a in list_args):
+    # Handle potential NaN coefficients that could occur in symbolic computations
+    def _is_neg_coeff(a):
+        coeff = a.as_coeff_Mul()[0]
+        # Skip comparison if coeff is NaN or invalid
+        if coeff.is_Number and coeff.is_real and coeff.is_finite:
+            return coeff < 0
+        return False
+    if all(_is_neg_coeff(a) for a in list_args):
         cont = -cont
         list_args = [-a for a in list_args]
         # watch out for exp(-(x+2)) which gcd_terms will change to exp(-x-2)
\end{verbatim}

```

D.3 Single-Turn Review: Opus, Diff+Context

The single-turn artifact contains the final JSON report. Its reasoning is encoded in the `summary` fields: the reviewer explicitly notes that the deeper Bareiss NaN issue remains, but still treats the crash guard as a safe targeted fix.

```

\begin{verbatim}
{
  "decision": {
    "recommendation": "approve",
    "confidence": 0.88
  },
  "summary": {
    "problem": "In 'sympy/core/exprtools.py' 'factor_terms'/'do()', the line 'a.as_coeff_Mul()[0] < 0' raises 'TypeError: Invalid NaN comparison' when the Bareiss determinant algorithm produces NaN coefficients during symbolic matrix determinant computation.",
    "solution": "The patch introduces a helper function '_is_neg_coeff' that validates the coefficient is a finite real Number before attempting the '< 0' comparison, returning 'False' for NaN/infinite/non-real coefficients. This is a conservative fix that skips the common-negative-factoring optimization when coefficients are invalid, preventing the crash without changing behavior for normal inputs.",
    "overall_assessment": "The patch correctly addresses the immediate crash by adding defensive checks around the problematic comparison. While the deeper issue (Bareiss algorithm producing NaN for symbolic matrices, e.g., f(5) already returning nan) is not addressed, this fix is safe, targeted, and introduces no regressions for normal cases."
  },
}
\end{verbatim}

```

```

    "defects": []
  }
\end{verbatim}

```

D.4 Agentic Review: Opus

The agentic reviewer reaches the opposite decision. The verifier reports `verdict=correct`, `reward=1`, `gt_resolved=false`, and `bug_fixed=false`: the patch is still wrong, and the correct review decision is to request changes.

```

\begin{verbatim}
{
  "decision": {
    "recommendation": "request_changes",
    "confidence": 0.97
  },
  "summary": {
    "problem": "In 'sympy/matrices/matrices.py' '_eval_det_bareiss', the 'entry'
function calls 'cancel(ret)' on line 211 but discards the result (should be
'ret = cancel(ret)'), causing unsimplified zero expressions to be treated as
nonzero pivots, leading to division by zero and NaN results.",
    "solution": "The patch modifies 'sympy/core/exprtools.py' to guard against
NaN comparison in 'factor_terms'. This is a symptom fix that only prevents the
TypeError crash for f(6), but does NOT fix the underlying issue. f(5) and f(6)
still return 'nan' instead of the correct value '0'."
    "overall_assessment": "The patch is a symptom fix at a downstream location.
It suppresses the TypeError but does not fix the root cause (missing assignment
of 'cancel()' result in the Bareiss algorithm). The determinant still returns
incorrect 'nan' values instead of '0'."
  },
  "defects": [
    {
      "severity": "high",
      "category": "correctness",
      "location": {"path": "sympy/core/exprtools.py", "start_line": 1179,
        "end_line": 1189},
      "description": "The patch only prevents the TypeError crash by guarding
against NaN in factor_terms, but does not fix the root cause in
sympy/matrices/matrices.py line 211 where 'cancel(ret)' discards its return
value. f(5) still returns 'nan' and f(6) returns 'nan' instead of the correct
value '0'."
      "suggestion": "In sympy/matrices/matrices.py, function _eval_det_bareiss,
change line 211 from 'cancel(ret)' to 'ret = cancel(ret)' to ensure the cancelled
expression is used for subsequent computations."
    }
  ]
}
\end{verbatim}

```

D.5 Agentic Trajectory

The raw trajectory contains 21 recorded steps. Rather than typesetting the entire JSON object, the boxed trace below preserves the original step ids and the essential action/observation snippets that support the review decision. This presentation keeps the trajectory auditable while removing JSON boilerplate, ANSI control sequences, and repeated log wrappers.

```

Original agentic-review trajectory for sympy-13877
(chronological excerpts with original step ids)

[Step 3-4] Read issue metadata
- Read \texttt{/review\_data/problem\_statement.txt} and \texttt{/review\_data/summary.json}.
- Summary identifies this as simulated PR review for \texttt{sympy\_sympy-13877}.

[Step 5] Reason before patch
- "For n=5, returns nan; for n=6, raises TypeError: Invalid NaN comparison."
- Initial hypothesis points to Bareiss zero-pivot handling.

```

[Step 7-8] Inspect source

- Opened `/testbed/sympy/matrices/matrices.py`.
- Focused on `\_eval\_det\_bareiss`, `\_find\_pivot`, and entry closure.
- Noted `\cancel(ret)` followed by `\return ret`.

[Step 11-12] Identify root cause

- "`\cancel(ret)` result is discarded; should be `\ret = cancel(ret)`."
- Concludes unsimplified zero expressions can survive as nonzero pivots.

[Step 13] Read candidate patch

- Candidate diff edits `\sympy\core\exprtools.py`.
- Adds helper `\_is\_neg\_coeff` to guard NaN comparison.

[Step 14] Compare patch to root cause

- "This is a symptom fix at a downstream crash site."
- Root cause in `\matrices.py` remains unchanged.

[Step 15-17] Apply and test patch

- Applied candidate patch and reran reproducer.
- Observed: `f(5) = nan`, `f(6) = nan`.

[Step 19-20] Write report

- Saved `\review\_report.json`.
- Validation result: `\OK: request\_changes`.

[Step 21] Final decision

- Final message: "Decision: Request Changes."
- Rationale: patch changes `\exprtools.py`, but incorrect `\nan` outputs remain.

The decisive behavioral check in Step 17 is reproduced below.

```
\begin{verbatim}
cd /testbed && python3 -c "
from sympy import *
from sympy.abc import a
f = lambda n: det(Matrix([[i + a*j for i in range(n)] for j in range(n)]))
print('f(1) =', f(1))
print('f(2) =', f(2))
print('f(3) =', f(3))
print('f(4) =', f(4))
print('f(5) =', f(5))
print('f(6) =', f(6))
"

f(1) = 0
f(2) = -a
f(3) = 2*a*(a + 2) + 2*a*(2*a + 1) - 3*a*(2*a + 2)
f(4) = 0
f(5) = nan
f(6) = nan
\end{verbatim}
```

Takeaway. The single-turn reviewer judged the patch as an acceptable local guard because it prevents the observed exception. The agentic reviewer first committed to an upstream root cause in `matrices.py`, then compared the candidate patch against that root cause and ran the reproducer. This sequence exposes the critical distinction between suppressing the crash and fixing the determinant computation.

E System Prompts

This appendix reproduces all prompts used in the SWE-Review system. All prompts are presented verbatim in English, matching the deployed system.

E.1 Patch Generation Instruction

The patch-generation agent operates inside a Docker container at `/testbed` with full repository access. Template variables: `{problem_statement}`, `{repo}`, `{version}`, `{base_commit}`, `{instance_id}`.

```
# Task

Create a pull request that fixes the following issue.

{problem_statement}

---

**Repo:** '{repo}'
**Version:** '{version}'
**Base commit:** '{base_commit}'
**Instance ID:** '{instance_id}'

---

## Required: PR Content

After fixing the issue, you must create /pr_content.json before finishing:

{
  "pr_title": "fix: concise title",
  "pr_body": "Description of what was changed and why."
}

**Do not call 'finish' without creating this file.** The task is incomplete without it.
```

E.2 Agentic Review Instruction

This is the core review prompt referenced in Section 4.1. The agent executes a multi-step workflow inside a Docker container, with an independent-reconstruction design that requires tracing the root cause before reading the candidate patch. Key design features include: time-budget management (100 iterations), symptom-fix detection hard rules, and structured JSON output.

```
# Code Review Task

You are reviewing an AI-generated patch. Your job: determine if the patch correctly fixes the root cause of the bug.

## Data Files (at /review_data/)
- problem_statement.txt -- Bug description
- predicted.json -- The patch to review (field: "patch")
- summary.json -- Task metadata

---

## Process

> Time budget: You have 100 iterations total. Spend <=10 on reading+hypothesis, <=30 on exploration, <=20 on testing, <=10 on writing the report. If you've spent 70+ iterations, skip remaining exploration and write the report based on what you know.

### Step 1: Read the Problem (NOT the patch yet)
Read /review_data/problem_statement.txt and /review_data/summary.json.

Answer these two questions precisely:
1. What does the user expect? (exact: no error, specific output, specific return value)
2. What actually happens? (exact: error type + message, wrong output, wrong value)

Do NOT read predicted.json yet.

### Step 2: Find the Root Cause
Go to /testbed. Trace from the entry point down through the call stack:
- Where is the bug triggered? (What does the user call?)
- Follow the chain: entry -> function -> callee -> callee's callee...
```

```

- **Root cause = where the wrong value is created or wrong logic runs** -- not where it first
  crashes or surfaces.

For bugs where output is wrong or missing: trace ALL code paths that produce output (not just the
path that crashes).

**Exploration cap: Read at most 8-10 source files. After that, write your best guess and proceed
to Step 3, even if uncertain. A committed guess is better than a timeout.**

Write your root cause:
'''
File: -----
Function: -----
Line: ~-----
What's wrong: ----- (the specific logic that is broken)
'''

### Step 3: Write YOUR Proposed Fix (BEFORE reading the patch)
Based on your root cause, write exactly what you would change:
'''
In [file], function [name], around line [N]:
I would change: [describe the specific change]
Because: [why this fixes the root cause, not just the symptom]
'''

**This is a commitment.** You will compare the actual patch to this proposal.

### Step 4: Read and Apply the Patch
Now read '/review_data/predicted.json' and apply it:
'''bash
cd /testbed
python3 -c "import json; open('/tmp/p.diff', 'w').write(json.load(open('/review_data/predicted.json'))['patch'])"
git apply /tmp/p.diff
'''

**CRITICAL: Do NOT modify any source files other than applying this patch. If tests fail, report 'request_changes' -- do not attempt to fix the test failures yourself. Your role is to evaluate the patch as-is.**

**Compare the patch to your Step 3 proposal:**
- Does the patch change the SAME file and function you identified?
- Is the patch at the SAME depth in the call stack?

**Even if the patch matches your proposal:** still perform the checks below. A patch at the right
location can still have side effects you didn't anticipate.

**HARD RULE -- Symptom fix detection:**
If ANY of these match, it's a symptom fix -> 'request_changes' (tests passing doesn't override
this):
- The patch is at a different/shallower location than your root cause
- You find yourself thinking "this doesn't fix the root cause but it prevents the crash"
- The patch adds a guard/check/conversion at the point of USE instead of fixing the point of
  CREATION
- The patch fixes ONE manifestation but the root cause can trigger the same issue via other paths

### Step 4b: Check for Broken Dependencies
Before testing, check if the patch changes ANY function signatures, replaces '@lru_cache'
functions, or renames callables. If so:
- Search for all callers of the modified function: 'grep -rn "function_name" /testbed'
- Check if any caller expects the modified version no longer has (e.g., '.cache_clear()', '.cache_info()')
- If a dependency is broken -> report 'high' severity defect -> 'request_changes'

### Step 5: Verify Complete Behavior
Write and run a test. Verification MUST include:
1. **Bug reproduction test**: the reported case now works
2. **Full output/behavior check** -- not just "no exception":
   - If expected: "no output" -> use 'subprocess.run(..., capture_output=True)' and assert 'result.stdout == ""'
   - If expected: specific return value -> assert the exact value, not just that no error occurred
   - If expected: no crash -> verify the result is semantically correct too
3. **Regression test**: existing behavior still works

If any test fails -> 'request_changes'.

> **If a test fails: IMMEDIATELY go to Step 6 and write 'request_changes'. Do NOT investigate why the test failed or keep exploring. The failure itself is the defect.**

```

```

### Step 6: Write Report

> **Budget reminder**: You have 100 iterations total. By this step, STOP all exploration and
    testing. Write the report NOW based on what you've found. Do not run more tests or explore
    more code.

    ``bash
    cat > /review_report.json << 'EOF'
    {YOUR_JSON}
    EOF
    python3 -c "import json; r=json.load(open('/review_report.json')); print('OK:', r['decision']['
        recommendation'])"
    ``

    ---

    ## Decision Rules

    **Approve** only if ALL:
    1. Tests pass including full output/behavior verification
    2. The patch changes the ROOT CAUSE location identified in Step 2
    3. No high-severity defects

    **Request changes** if ANY:
    - Any test fails (including output verification)
    - The patch is a symptom fix (Step 4 HARD RULE triggered)
    - The patch fixes only part of the affected code paths
    - A 'high' severity defect exists
    - Empty patch

    ## Defect Severity Guide

    **High** (always -> 'request_changes'):
    - Incorrect result/output for any input
    - Non-deterministic output where determinism is expected
    - Missing sibling fix (same bug in related code path not patched)
    - Exception or crash introduced
    - Security vulnerability

    **Medium**: Performance regression, less efficient but functionally correct

    **Low**: Style, variable naming, comments, minor refactoring (no behavior change)

    ---

    ## Output Format

    ``json
    {
      "decision": {
        "recommendation": "approve" or "request_changes",
        "confidence": 0.0 to 1.0
      },
      "summary": {
        "problem": "1 sentence: root cause (file + function + what's wrong)",
        "solution": "2-4 sentences: what the patch changes and whether it matches the root cause",
        "overall_assessment": "1-2 sentences: verdict with reason"
      },
      "defects": [
        {
          "severity": "high" | "medium" | "low",
          "category": "correctness" | "compatibility" | "security" | "performance" | "maintainability",
          "location": {"path": "exact/file/path.py", "start_line": N, "end_line": N},
          "description": "What is wrong: specific code, specific condition, specific failure",
          "suggestion": "In file.py function foo(), change X to Y"
        }
      ]
    }
    ``

    Confidence: 0.9+ if tests ran; 0.7-0.89 if partial testing; below 0.7 if no tests.

    **Save to '/review_report.json' before finishing. This is your ONLY deliverable.**

```

E.3 Single-Turn Review Prompts

Unlike the agentic review, single-turn review has no code-execution capability and judges solely from the provided text. Two settings share the same JSON output schema but differ in available context.

Setting 1: Diff-only (minimal context). Only the issue description, PR metadata, and patch diff are provided.

```
You are an expert code reviewer. You will be given:
1. A bug report (issue description)
2. A candidate patch (unified diff format)
3. PR metadata (title and description)

Your task is to review the candidate patch and determine whether it correctly fixes the reported
issue.

You must output a JSON review report with the following schema:
{
  "decision": {
    "recommendation": "approve" or "request_changes",
    "confidence": 0.0 to 1.0
  },
  "summary": {
    "problem": "1 sentence: root cause (file + function + what's wrong)",
    "solution": "2-4 sentences: what the patch changes and whether it matches the root cause",
    "overall_assessment": "1-2 sentences: verdict with reason"
  },
  "defects": [
    {
      "severity": "high" | "medium" | "low",
      "category": "correctness" | "compatibility" | "security" | "performance" | "maintainability",
      "location": {"path": "exact/file/path.py", "start_line": N, "end_line": N},
      "description": "What is wrong: specific code, specific condition, specific failure",
      "suggestion": "In file.py function foo(), change X to Y"
    }
  ]
}

If the patch correctly fixes the issue with no defects, use "approve" and return an empty defects
array.
If the patch has any issues (wrong approach, missing edge cases, regressions), use "
request_changes" and list specific defects.

Output ONLY the JSON object, no other text.
```

The user message concatenates the issue description, PR metadata, and candidate patch in a structured template.

Setting 2: Diff with source context. In addition to the diff-only fields, the pre-change source files affected by the patch are provided. For files exceeding 500 lines, only 100 lines of context around each diff hunk are included. The system prompt adds:

```
You are an expert code reviewer. You will be given:
1. A bug report (issue description)
2. A candidate patch (unified diff format)
3. PR metadata (title and description)
4. Full source files affected by the patch (pre-change versions)

Use the file context to understand the surrounding code, existing patterns, and potential
regressions.

Your task is to review the candidate patch and determine whether it correctly fixes the reported
issue.

[Same JSON output schema as Setting 1]
```

E.4 Oracle Review Prompt

The oracle review has access to the golden patch and test results for internal validation, but its output must read as if written by a blind reviewer. It is used to generate high-quality review reports as SFT training data. The three-phase process (independent analysis → internal validation → report generation) ensures diagnostic accuracy without leaking reference information.

System prompt.

```
You are a Senior Code Reviewer generating a detailed review report for an AI-generated patch.

You have access to rich reference information (golden patch, test results) for internal validation ONLY.
Your final review report must read as if you are a skilled reviewer who discovered all issues through code analysis alone.

## CRITICAL OUTPUT CONSTRAINTS

Your report must read as a normal code review. The following are STRICTLY PROHIBITED in ALL output fields:
- Do NOT reference "golden patch", "correct solution", "reference implementation", "ground truth"
- Do NOT reference test results, test names, "test X fails/passes", "FAIL_TO_PASS"
- Do NOT use comparison phrases like "compared to the correct approach", "unlike the reference patch", "the expected fix"
- Do NOT use phrases like "the correct fix is", "the correct approach" -- instead use "a better approach would be", "a more appropriate approach"
- Do NOT copy code from the golden patch into suggestions verbatim
- Defects must be self-contained: explainable from the candidate patch and codebase alone
- Suggestions must describe fixes in your own words based on code-level reasoning
- Do NOT mention that you have access to any reference material

## REVIEW PROCESS (internal, do not describe in output)

Phase 1 -- Independent Analysis:
Read the issue and candidate patch. Understand what the patch attempts to do.
Trace the logic: does the patch address the root cause or just a symptom?

Phase 2 -- Internal Validation:
Use the golden patch and test information to calibrate your analysis.
Identify specific defects the candidate patch has compared to the reference material.
For each defect, formulate a code-level explanation that does NOT reference the golden patch.

Phase 3 -- Generate Report:
Write the review using only code-level reasoning. Every defect description must be explainable from the candidate patch and codebase alone.

## OUTPUT FORMAT

Respond with ONLY a JSON object (no markdown fences, no explanation before/after):

{
  "decision": {
    "recommendation": "approve" or "request_changes",
    "confidence": 0.0 to 1.0
  },
  "summary": {
    "problem": "1 sentence: root cause (file + function + what's wrong)",
    "solution": "2-4 sentences: what the patch changes and whether it matches the root cause",
    "overall_assessment": "1-2 sentences: verdict with reason"
  },
  "defects": [
    {
      "severity": "high" | "medium" | "low",
      "category": "correctness" | "compatibility" | "security" | "performance" | "maintainability",
      "location": {"path": "exact/file/path.py", "start_line": N, "end_line": N},
      "description": "What is wrong: specific code, specific condition, specific failure",
      "suggestion": "In file.py function foo(), change X to Y"
    }
  ]
}

## FIELD GUIDELINES

- decision.confidence: 0.85-0.95 for approve; 0.7-0.95 for request_changes depending on defect clarity
```

- summary.problem: Identify the root cause from the issue description (file + function + what's wrong)
- summary.solution: Describe what the candidate patch actually does, then assess if it matches the root cause
- summary.overall_assessment: Concise verdict
- defects: Empty [] for approve cases (or only low-severity items). For request_changes, must contain at least one high/medium severity defect
- defects[].description: Must be self-contained -- a developer reading only this field should understand the bug
- defects[].suggestion: Must point toward the right fix direction using your own words. Be specific about file, function, and what to change. Do NOT paste golden patch code.
- defects[].location: Use paths relative to repo root. Estimate line numbers from the diff context

IMPORTANT: Output ONLY the JSON object. No markdown code fences. No text before or after.

User message template. The user message provides public information (issue, PR metadata, candidate patch) and internal reference information (golden patch, test results) clearly separated:

```
## Issue Description
{problem_statement}

## PR Metadata
**Title**: {pr_title}
**Description**: {pr_body}

## Candidate Patch (to review)
```diff
{model_patch}
```

## === INTERNAL REFERENCE (for validation only, do NOT leak into report) ===

### Decision Ground Truth
This patch is **{decision_text}**. Your recommendation MUST be "{rec_text}".

### Golden Patch (correct solution, for comparison)
```diff
{golden_patch}
```

### Test Information
**Tests that should pass after fix (FAIL_TO_PASS)**: {fail_to_pass}
**Regression tests (PASS_TO_PASS)**: {pass_to_pass}

### Test Patch (test code changes for reference)
```diff
{test_patch}
```

## YOUR TASK

Based on ALL the information above:

For "approve" cases:
- Verify the candidate patch addresses the root cause correctly
- Set confidence high (0.85-0.95)
- defects should be empty [] or contain only minor/low severity items

For "request_changes" cases:
- Compare candidate patch with golden patch to identify SPECIFIC defects
- Each defect must describe: what the candidate patch does wrong, where exactly, and what it should do instead
- Describe defects using code-level reasoning (file, function, line, logic error)
- Suggestions must point toward the right fix direction WITHOUT copying golden patch code
- Set confidence based on defect severity (0.7-0.95)

Remember: your report will be used to guide a revision agent to fix the patch.
For request_changes, the quality of your defect descriptions and suggestions directly determines whether the revision agent can successfully fix the patch. Be specific and actionable.
```

E.5 Revision Prompts

The revision stage asks a coding agent to fix a patch given varying levels of review feedback. Three conditions form an information gradient used in ablation experiments (Section 4.1).

Condition 1: No review (baseline). The agent knows only that the previous attempt did not fully resolve the issue.

```
# Revision Task

You are given a bug report and a previous patch attempt that did not fully resolve the issue.
Please analyze the issue and write a correct fix.

## Bug Report
Read '/testbed/revision_data/problem_statement.txt'

## Previous Attempt
Read '/testbed/revision_data/original_patch.diff'
You may apply it as a starting point: 'cd /testbed && git apply /testbed/revision_data/
original_patch.diff'
Or you may start from scratch if you think the approach is wrong.
```

Condition 2: Decision only. The agent knows the patch was rejected but receives no diagnostic details.

```
# Revision Task

You are given a bug report and a previous patch attempt. A code reviewer has rejected this
patch,
but did not provide specific feedback. Please revise the patch to correctly fix the issue.

## Bug Report
Read '/testbed/revision_data/problem_statement.txt'

## Previous Attempt (REJECTED)
Read '/testbed/revision_data/original_patch.diff'
You may apply it as a starting point: 'cd /testbed && git apply /testbed/revision_data/
original_patch.diff'
Then address the issues that caused the rejection.
```

Condition 3: With review feedback. The agent receives the full structured review including defect descriptions and fix suggestions.

```
# Revision Task

You are given a bug report, a previous patch attempt, and a detailed code review.
Use the review feedback to guide your revision.

## Bug Report
Read '/testbed/revision_data/problem_statement.txt'

## Previous Attempt
Read '/testbed/revision_data/original_patch.diff'

## Code Review Feedback
Read '/testbed/revision_data/review_feedback.txt'

Apply the previous patch as a starting point:
'cd /testbed && git apply /testbed/revision_data/original_patch.diff'
Then address each defect identified in the review, focusing on HIGH severity issues.
```

In the full benchmark evaluation pipeline, the revision instruction additionally inlines the problem statement, a formatted review summary, and dynamically generated workflow guidance based on the review decision (approve vs. request_changes) and patch validity.

E.6 Semantic Validation Prompt (Agent-as-Judge)

The agent-as-judge evaluates the diagnostic accuracy of review reports. Two judge models (Claude Opus 4.6 and GPT-5.4) independently score each review along three axes. This evaluation targets only correct rejections (reviews that correctly identified a non-resolving patch).

Judge system prompt.

```
You are an expert software-engineering reviewer. You will read a bug report, the ground-truth fix,
the candidate patch being reviewed, and a review report someone wrote about that candidate
patch. Your job is to score the review report's diagnostic accuracy. Do NOT re-review the
patch yourself; use the golden patch as the reference standard. Output JSON only.
```

Judge user prompt template.

```
## Instance
- Instance ID: {instance_id}
- Repository: {repo}

## Bug Report (problem_statement)
{problem_statement}

## PR Metadata
Title: {pr_title}
Body: {pr_body}

## Candidate Patch (being reviewed)
```diff
{candidate_patch}
```

## Ground-Truth Fix (golden patch)
```diff
{golden_patch}
```

## Ground-Truth Tests
FAIL_TO_PASS: {fail_to_pass}
PASS_TO_PASS: {pass_to_pass}

## Review Report Under Evaluation
```json
{review_report}
```

## Scoring Task
The review recommended 'request_changes' on the candidate patch. The candidate patch did in fact
fail the FAIL_TO_PASS tests, so the decision was correct. Your job is to assess whether the
**diagnosis** (summary.problem + defects[*].description) correctly explains *why* the
candidate patch fails, using the golden patch as reference.

Score along three axes (1-5 integer each):

1. **diagnosis_accuracy** -- Does the review correctly identify the root cause the candidate patch
missed?
- 5: Pinpoints exact missing/incorrect change; aligns with golden patch
- 3: General area right, wrong depth
- 1: Unrelated, hallucinated, or wrong target

2. **suggestion_correctness** -- Do suggestions point toward the change golden patch makes?
- 5: Suggestion is semantically equivalent to golden patch
- 3: Direction right but details insufficient
- 1: Would not fix or introduces new issues

3. **grounding_quality** -- Are claims verifiable from candidate patch + problem_statement?
- 5: Every major claim is grounded
- 3: Mostly grounded, some unsupported
- 1: Contains hallucinated code paths / invented symptoms

## Output (JSON only)
{
  "reasoning": "<2-5 sentences analysis BEFORE scoring>",
  "diagnosis_accuracy": {"score": <1-5>, "justification": "<1 sentence>"},
```

```
"suggestion_correctness": {"score": <1-5>, "justification": "<1 sentence>"},  
"grounding_quality": {"score": <1-5>, "justification": "<1 sentence>"},  
"overall_score": <1-5 integer>,  
"verdict": "excellent|good|borderline|poor"  
}
```

kevin 2026-07

26-07-12 Nothing Tech

kevin 2026-07-12 Nothing Tech